

LINE: Queueing Analysis Algorithms

A Primer: Python Edition

Last revision: October 4, 2026
QORE Lab, Imperial College London

Contents

1	A guided tour of LINE	1
1.1	Overview and installation	1
1.1.1	Obtaining and installing LINE	1
1.1.2	Software requirements	1
1.1.3	Checking the installation	2
1.2	A first model	2
1.2.1	The M/M/1 queue	3
1.2.2	Reading the results	3
1.3	Network models	4
1.3.1	Nodes, scheduling and classes	4
1.3.2	Distributions	4
1.3.3	Open and closed networks	4
1.3.4	Routing and advanced features	5
1.4	Analysis	5
1.4.1	Performance metrics	5
1.4.2	Solvers	6
1.5	Layered queueing networks	7
1.6	Random environments	8
1.7	Further resources	8
2	Tutorials	10
2.1	Tutorial 1: A M/M/1 queue	10
2.2	Tutorial 2: A multiclass M/G/1 queue	12
2.3	Tutorial 3: Machine interference problem	14
2.4	Tutorial 4: Round-robin load-balancing	16
2.5	Tutorial 5: Modelling a re-entrant line	17
2.6	Tutorial 6: A queueing network with caching	19
2.7	Tutorial 7: Delayed-hit caches with a retrieval system	21
2.8	Tutorial 8: Response time distribution and percentiles	23
2.9	Tutorial 9: Optimizing a performance metric	24
2.10	Tutorial 10: Studying a departure process	25
2.11	Tutorial 11: Basic layered queueing network	27
2.12	Tutorial 12: Random environments	28
2.13	Tutorial 13: Posterior analysis	29
2.14	Tutorial 14: Open cluster	30
2.15	Tutorial 15: A loss network	32
2.16	Tutorial 16: A queueing Petri net	34
2.17	Tutorial 17: A colored generalized stochastic Petri net	35

Chapter 1

A guided tour of LINE

1.1 Overview and installation

LINE is an open-source software package for the analysis of queueing models by means of analytical methods and simulation. It is developed by the QORE lab at Imperial College London and distributed under the BSD-3 license. The package solves single queueing systems ($M/M/1$, $M/M/k$, $M/G/1$, ...), open and closed queueing networks, layered queueing networks, and models embedded in random environments. Models are solved either by LINE's native algorithms or through external solvers such as JMT and LQNS.

LDES, the discrete-event simulation engine of LINE, is built to span the entire feature space of the package, so that every approximation LINE publishes can be validated against a sample path of the same model, expressed once. Several analytical solvers are included alongside it to accelerate the evaluation of specific classes of models, and their solutions can in turn warm-start an LDES run, placing it near steady state already at the start of the simulation (Sec. 1.4.2).

In this primer, we give a guided tour of the package. We first build and solve a single queue, so as to introduce the modeling workflow and the format of the results. We then generalize the workflow to arbitrary network models and survey the available solvers. Lastly, we introduce layered queueing networks and models embedded in random environments. Advanced topics, such as transient and distribution analysis, fall beyond the scope of this primer and are treated in the user manual.

1.1.1 Obtaining and installing LINE

LINE is distributed on PyPI and installs with `pip`:

```
pip install line-solver
```

The analytical solvers and modelling API are native Python and require Python 3.11 or later. The LDES wrapper first looks for a native `ldes` executable; when an installation does not contain one, it downloads and runs `ldes.jar`. That fallback, and the JMT wrapper, require a Java runtime.

The latest releases and the source code are available from <http://line-solver.sf.net>.

1.1.2 Software requirements

The package requires CPython 3.11 or later, and `pip` installs NumPy, SciPy, pandas, Matplotlib, NetworkX, websockets and the MCP Python package along with it. Its modelling API and analytical solvers are native Python. LDES uses a native `ldes` executable when one is installed and otherwise downloads and runs `ldes.jar`; JMT and `lang='java'` also start Java. Thus a Java runtime is required for those paths. The Java archives (`JMT.jar`, `ldes.jar` and `jline.jar`) are excluded from the distribution to keep it small and are downloaded on first use. Two extras are available: `pip install`

`line-solver[symbolic]` adds SymPy, which backs symbolic analysis when the SageMath backend described below is absent, and `pip install line-solver[native]` adds PyTorch, an optional tensor backend that enables GPU execution and automatic differentiation. Jupyter, or an equivalent, is needed only to open the `.ipynb` examples.

Three external tools are optional. Each one backs a specific part of LINE and none of them is needed by the native solvers:

- *Java Modelling Tools* (JMT, <http://jmt.sf.net>), version 1.3.0 or later, is required by the JMT simulation solver. It is downloaded automatically on the first call to the solver.
- *LQNS* (<https://github.com/layeredqueuing/V6>), version 6.2.28 or later, is required by the LQNS wrapper, for layered models and for the `qnsolver` methods it offers on flat networks. The `lqns`, `lqsim` and `qnsolver` commands must be on the system path.
- The *symbolic backend*, a SageMath computer algebra system behind a small REST service, is used by the symbolic methods of the CTMC and fluid solvers. It requires Docker and is fetched once with `docker pull imperialqore/line-sage-rest:latest`, after which LINE starts and stops the service itself. With the `symbolic` extra installed, symbolic analysis falls back on SymPy when this service is absent; a base installation contains neither symbolic backend.

The user manual gives the full list, with the version constraints and the platform-specific installation commands.

1.1.3 Checking the installation

Every distribution ships an environment check that reports which optional dependencies (a Java runtime, the LQNS binaries, JMT, the C++ `line-cli` engine, and the SageMath symbolic backend) are reachable. It reports only: a missing JMT is named, not downloaded, so that running the check does not acquire the dependency it is reporting on. Run it once after installing:

```
line-install
```

On a machine where every dependency is reachable, the check reports:

```
Checking Java...
Checking LQNS...
Checking JMT...
  /path/to/line/common/JMT.jar
Checking C++ engine (line-cli)...
  /path/to/line/common/line-cli
Checking symbolic backend (line-sage-rest)...
Success. LINE is ready to use.
```

A missing dependency is reported as a warning rather than an error, because it disables a subset of the solvers and leaves the rest of LINE usable. External services and engines are otherwise resolved by the method that requests them: symbolic methods connect to the configured SageMath backend, the JMT wrapper resolves a Java runtime and `JMT.jar`, and the LQNS wrapper requires the corresponding commands on `PATH`.

1.2 A first model

Every LINE model is assembled from the same four conceptual blocks: (1) the *nodes* of the network, among them the *stations* where jobs queue and are served; (2) the *job classes*, together with the *arrival* and *service* processes attached to each station and class; (3) the *topology*, that is, the routes the jobs follow; and (4) the *solution*, in which the finished model is handed to a *solver* that returns performance

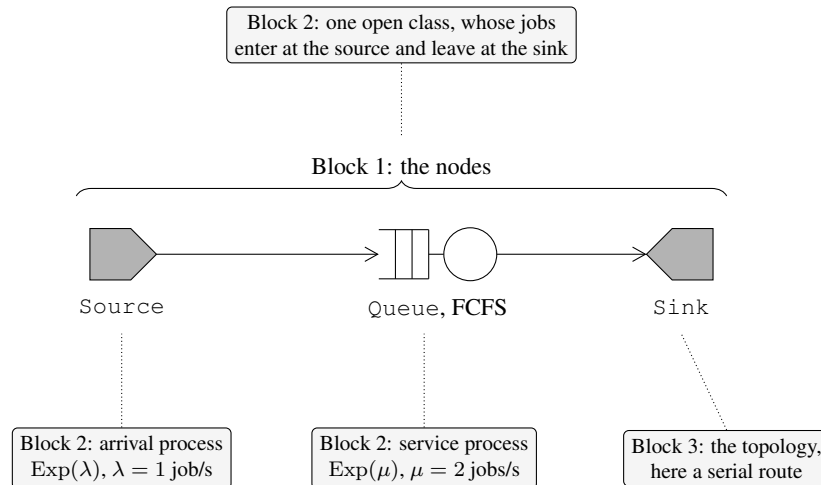


Figure 1.1: The M/M/1 queue, annotated with the blocks every LINE model is built from. A Poisson stream of rate λ feeds one FCFS station with exponential service of rate μ , and completed jobs leave through the sink. The fourth block, the solution, is the choice of solver and has no counterpart in the diagram.

metrics. The tutorial listings develop those blocks in reading order; the directly runnable companion files retain ordinary program structure and do not all repeat the four labels as comments. This chapter presents the concepts and the vocabulary that go with them; the scripts that realize them in the language of this edition are collected in Chapter 2, one worked model per section.

1.2.1 The M/M/1 queue

The smallest model worth building is the M/M/1 queue: a single first-come first-served station, fed by a Poisson arrival stream, whose service times are exponentially distributed. Figure 1.1 shows it with the first three blocks named on the diagram. The nodes are a `Source`, a `Queue` and a `Sink`; a single open class of jobs enters at the source and leaves at the sink, arriving at rate $\lambda = 1$ and being served at rate $\mu = 2$; and the topology is the serial route that connects the three nodes in order. The fourth block, the solution, does not appear in the diagram: it is the choice of solver.

With these parameters elementary queueing theory gives a utilization of $\rho = \lambda/\mu = 0.5$ and a mean response time of $1/(\mu - \lambda) = 1\text{s}$. An analytical LINE solver returns those values exactly; the LDES simulation in Tutorial 1 estimates them within sampling error. The script that builds and solves this model is Tutorial 1 (Section 2.1).

1.2.2 Reading the results

Every average-metric solver returns a table with one row per (station, class) pair. The main columns are as follows:

Column	Meaning
QLen	mean number of jobs at the station (queue length)
Util	fraction of the servers kept busy; at a delay (infinite-server station), the mean number in service
RespT	mean response time per visit
ResidT	mean residence time (response time weighted by visits)
ArvR	mean arrival rate
Tput	mean throughput

A row is retrieved by the station and class *objects* that produced it, or by their names, rather than by position, so that a script never depends on the order in which the nodes were declared; filtering by a

station alone, or by a class alone, returns every row that mentions it, and a whole column can be read out as a plain list. The selection idioms of this edition are shown in Section 2.1.

Stochastic solvers additionally accept a `seed`, which makes a run reproducible, and a `samples budget`, which trades accuracy against time. Verbosity is controlled globally through `GlobalConstants`.

1.3 Network models

We now generalize the workflow of Section 1.2 to arbitrary networks. The `Network` object is the container for all model elements, which register themselves with the model passed to their constructor. The four-block structure carries over unchanged; only the number of nodes, classes, and routes grows.

1.3.1 Nodes, scheduling and classes

Node	Role
Source / Sink	external arrival / departure (open classes)
Queue	queueing station with a scheduling discipline
Delay	infinite-server (pure think time / delay)
Router	pure routing / load-balancing node
ClassSwitch	changes the class of transiting jobs
Cache	caching node (hit/miss with a replacement policy)
Fork / Join	fork-join parallelism
Place / Transition	stochastic Petri-net elements

The scheduling discipline is passed to the `Queue` constructor as a `SchedStrategy` value: FCFS (first-come first-served), PS (processor sharing), INF (infinite server), LCFS, SIRO (random order), HOL (head-of-line priority), SEPT/LEPT, and DPS/GPS. A queue's server count is set with `setNumberOfServers`. Two class kinds exist: `OpenClass` jobs enter at a `Source` and leave at a `Sink`; `ClosedClass` jobs circulate among a fixed population with a designated reference station. Open and closed classes may coexist in a mixed model.

1.3.2 Distributions

Arrival and service processes are distribution objects: `Exp` (exponential), `Erlang`, `HyperExp`, `Cox2/APH` (phase-type), `Det` (deterministic), `Uniform`, `Gamma`, `Pareto`, `Lognormal`, `MAP/MMPP2` (Markovian arrivals), and `Replayer` (empirical trace). Non-exponential timing is handled by phase-type expansion or by fitting: `APH.fit_mean_and_scv(m, scv)` matches a target mean and squared coefficient of variation.

1.3.3 Open and closed networks

A network is *open* when jobs enter it from a `Source` and leave it at a `Sink`, so that the population present at any instant is itself a random variable; it is *closed* when a fixed population circulates indefinitely, and *mixed* when open and closed classes coexist. A closed class names a *reference station*, which marks the point at which a job is considered to have completed one cycle through the network; system response time and system throughput are measured against it.

Figure 1.2 shows a closed network of the kind that recurs throughout queueing theory, the machine-repairman model: three machines alternate between a working state, modelled as a delay because a machine never waits to start working, and a repair station staffed by two repairmen, which is an ordinary FCFS queue with two servers. Machines fail at rate 0.5 and are repaired at rate 4. Because the state space is small, the model can be solved exactly by generating its Markov chain: on average 2.66 of the three machines are working and 0.34 are under repair, at a repair-station utilization of 0.167 per server. The corresponding script is Tutorial 3 (Section 2.3).

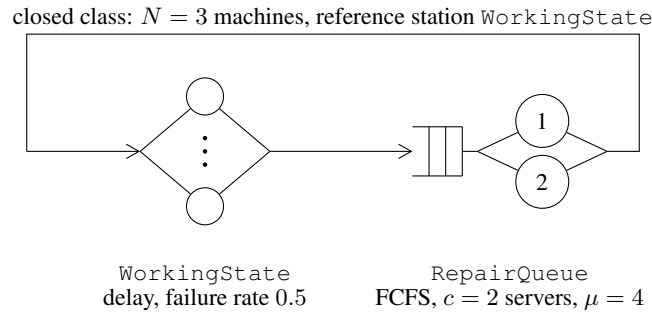


Figure 1.2: A closed network: the machine-repairman model. A fixed population of $N = 3$ machines alternates between a working state, modelled as a delay, and a two-server FCFS repair station. No job enters or leaves, so exactly N jobs are present at all times.

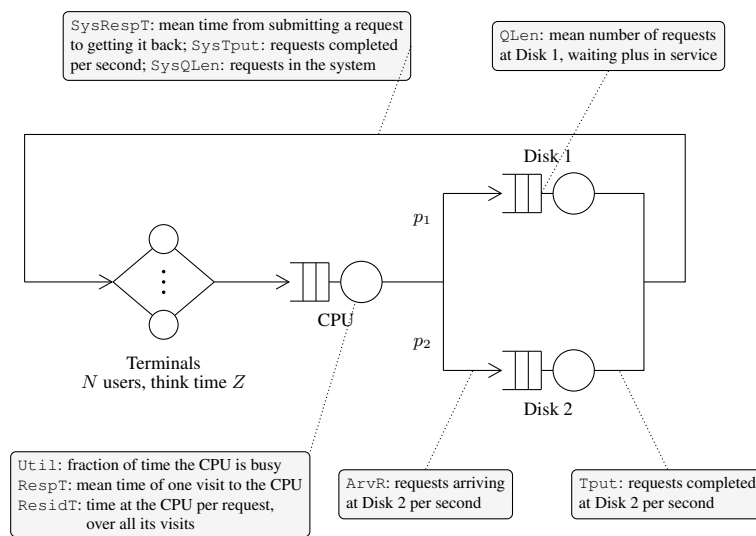


Figure 1.3: The metrics LINE reports, named on a closed network in which N users at terminals submit requests that are served by a CPU and by one of two disks before returning to their user. QLen, Util, RespT, ResidT, ArvR and Tput are reported per station and class; the system metrics are reported per chain.

1.3.4 Routing and advanced features

Serial chains are declared with a serial-routing shorthand; point-to-point links are added one at a time, and per-node behaviour is set with a `RoutingStrategy` such as `RAND`, `RROBIN` (round robin), `PROB` (probabilistic), or `JSQ` (join shortest queue). Class-dependent routing uses a routing matrix obtained from the model and populated per class. LINE also supports finite-capacity regions, reward models, caches with replacement policies, class switching, and stochastic Petri nets; their treatment falls beyond the scope of this primer and is given in the user manual.

1.4 Analysis

1.4.1 Performance metrics

The primary analysis is steady-state average performance. The metrics of Section 1.2 are reported per station and class; Figure 1.3 names them on a network, so that each can be read off the place in the model it refers to.

Response times, throughputs and queue lengths are also reported end-to-end. These are the *system* metrics `SysRespT`, `SysTput` and `SysQLen`, indexed by chain rather than by station-class pair: a system response time is the time a job spends in the network, measured from its arrival at the `Source` for an open chain or from its departure from the reference station for a closed one, until it reaches the `Sink` or returns to that station.

Beyond averages, LINE reports transient averages, response-time distributions and, for Markovian models, the underlying state space and generator, from which state probabilities are read; these fall beyond the scope of this primer and are covered in the user manual.

1.4.2 Solvers

A solver takes a model and a set of options and returns metrics. All solvers are invoked uniformly, as `Solver(model, options)` followed by the method that selects the metrics wanted, and since every solver consumes the same model object unchanged, running two solvers and cross-validating their results is good practice. Solvers fall into three groups. *Native solvers* implement an analysis algorithm inside LINE. *Meta-solvers* do not analyze a model themselves: they decompose or replicate it and delegate each piece to another solver, which the caller supplies. *Wrappers* translate the model into the input format of an external tool, run it as a separate process, and read its results back.

Solver	Method	Best for
Native solvers		
AG	agent-based decomposition	product-form models with synchronization
BA	bound analysis	guaranteed bounds, optimization loops
CTMC	continuous-time Markov chain	exact, small state spaces
FLD	fluid / mean-field ODE	large populations, transient
LDES	discrete-event simulation (native)	non-Markovian timing, widest features
MAM	matrix-analytic methods	MAP/PH open models
MVA	mean value analysis	product-form open/closed, fast
NC	normalizing constant	large closed populations
SSA	stochastic simulation (native)	general, any feature
Meta-solvers		
AUTO	automatic solver selection	when no solver is chosen by hand
ENV	random-environment blending	environment models (Sec. 1.6)
LN	layer decomposition	layered models (Sec. 1.5)
UQ	prior-weighted expansion	models with uncertain parameters
External solvers (wrappers)		
JMT	discrete-event simulation (JMT)	validation, general
LQNS	<code>lqns / lqsim / qnsolver</code>	layered models; product-form multiserver (<code>qns.*</code>)

A meta-solver is given the inner solver to use as an argument, so the same layered or environment model can be evaluated at very different costs by changing that one argument. Wrappers are available only when their engines can be resolved at run time: JMT requires a Java runtime and `JMT.jar`, while LQNS requires the separately distributed `lqns`, `lqsim` and `qnsolver` commands. Their absence does not affect the native solvers. Solver support is also model-dependent: “general” in this table is a selection guideline, not a guarantee that every feature is implemented by that solver in every language edition.

The same model can also be solved by LINE’s Java or C++ engine. The queueing-network solver constructors supported by the dispatch layer take a `lang` keyword naming the codebase that does the work, and nothing else in the script changes:

```
MVA(model).avg_table()           # native Python (default)
MVA(model, lang='java').avg_table() # solved by the JAR, jline.jar
MVA(model, lang='cpp').avg_table()  # solved by the C++ engine, line-cli
```

The model is serialized to JSON, the other codebase’s command-line front end solves it, and its answer is read back into the usual result object. `lang='java'` needs a Java runtime; `lang='cpp'` needs

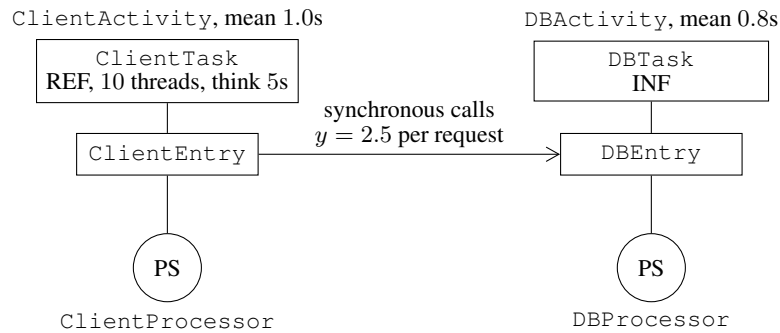


Figure 1.4: A layered queueing network: a two-tier client–server application. Each task offers one entry, runs one activity, and is hosted by its own processor; the client activity issues on average 2.5 synchronous calls per request to the database entry.

the `line-cli` binary and serves MVA, NC, CTMC, MAM, FLD, SSA, BA, AG, AUTO, JMT, LN and ENV. Meta-solvers whose constructor must receive an inner solver, notably ENV and UQ, do not themselves accept `lang`; set the language on the inner solver instead. Setting `LINE_SOLVER_LANG` switches the default for a whole session. Running one model under two engines is the cheapest cross-check available; the user manual documents the option in full.

As a rule of thumb, we recommend CTMC for small exact models; MVA/NC for product-form networks at scale; BA when a guaranteed interval is worth more than a point estimate; FLD for very large populations or transients; MAM for Markovian-arrival or phase-type open models; and SSA, LDES or JMT when a feature is not supported analytically, with LDES carrying the widest feature envelope of the three.

The two groups also combine. The solution of an analytical solver can be used to *warm-start* a simulation: the initial state of an LDES or SSA run is seeded with a job placement taken from a cheaper solver, so that the run starts near its steady state and no samples are spent traversing the initial transient. An exact auxiliary solver contributes the mode of its stationary distribution, an approximate one its rounded mean queue lengths, with the closed-class populations conserved. The placement is set with `init_from_solver`, and the *Warm start* section of the user manual documents which solver may seed which, and what the mechanism costs and buys.

1.5 Layered queueing networks

A `LayeredNetwork` (LQN) captures software contention on top of hardware contention. Its elements are *processors* (hardware servers), *tasks* (software resources with a multiplicity), *entries* (the service interfaces a task offers), and *activities* (units of work that issue synchronous or asynchronous calls to other entries). A synchronous call blocks the caller until the callee replies, so a task can be held by a resource it does not itself contend for – which is the phenomenon a flat queueing network cannot express.

Figure 1.4 shows a two-tier client–server application. The client task has ten threads, each thinking for 5s, doing 1s of its own work per request, and issuing on average 2.5 synchronous calls to the database entry; the database serves each call in 0.8s. Each task runs on its own processor-sharing processor.

The native LN solver decomposes the model into layers and applies a supplied per-layer solver, iterating until the layers agree; the external LQNS solver may be used instead when available. Results are reported per element rather than per station and class, one row for each processor, task, entry and activity, tagged with its type. A metric that is undefined for an element type is reported as NaN: processors carry no queue length or response time of their own, tasks report a residence time rather than a per-visit response time, and entries report the converse.

Solving this model with LN over MVA shows the database processor saturated at a utilization of 0.998, which is what caps the client throughput at 0.499 requests per second, against a client-side response time

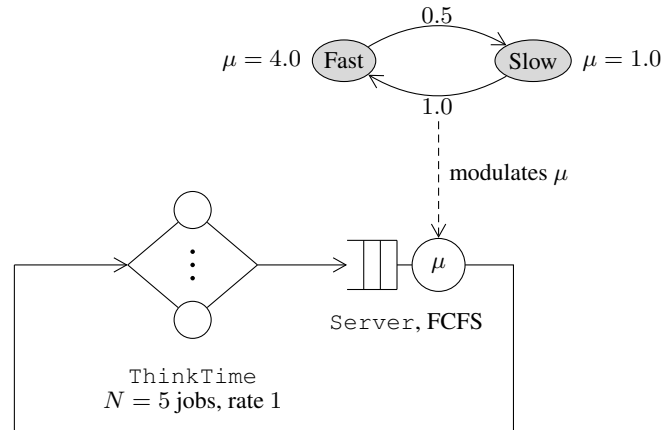


Figure 1.5: A model in a random environment: a two-state Markov chain alternates between a fast and a slow mode, each a complete stage network, and modulates the service rate of the queue. The environment is fast 2/3 of the time and slow 1/3 of the time.

of 15.0s per request. The script and its full result table are Tutorial 11 (Section 2.11).

1.6 Random environments

A random environment models a system whose parameters change according to an underlying stochastic process. Each *stage* is a complete network with its own parameters, and stage transitions follow specified distributions. The `ENV` meta-solver takes a factory producing an inner solver for each stage and returns metrics averaged over the stationary distribution of the environment.

Figure 1.5 shows the pattern at its simplest: a two-station closed network with five jobs, whose server alternates between a fast mode ($\mu = 4$) and a slow mode ($\mu = 1$). Only the service rate differs between the two stages; everything else is the same network. The environment leaves the fast stage at rate 0.5 and the slow stage at rate 1.0, so it spends 2/3 of the time fast and 1/3 slow, and the reported metrics are averages over that stationary distribution rather than over either stage alone. The script is Tutorial 12 (Section 2.12).

1.7 Further resources

The `line-cli.py` script solves models without writing code, reading LINE’s native JSON format as well as JMT’s JSIMG and LQNS’s LQNX, and printing tables, JSON, or CSV:

```
python line-cli.py solve example.json --solver mva
```

Every metric is reported on its own row, station by station and class by class, followed by the system metrics per chain. The same script also lists what is installed, each solver with the model formats it can read:

```
python line-cli.py list solvers
```

These commands refer to the script at the root of a source checkout. The installed `line` console uses its package interface; for example, `line -f example.json -s mva -a avg`. LINE is also available as a Model Context Protocol (MCP) server, letting LLM tools such as Claude Code build and solve models through natural language (`pip install line-solver`). The complete references are the LINE user manuals (LINE-user-matlab.pdf, LINE-user-java.pdf, LINE-user-python.pdf and LINE-user-cpp.pdf) with their Sphinx (MATLAB and

Python), Javadoc (Java) and Doxygen (C++) API references, plus the MCP getting-started guide, all linked from <http://line-solver.sf.net>.

Chapter 2 works through seventeen complete models in the language of this edition, beginning with the M/M/1 queue of Section 1.2. For many more worked examples, covering additional queueing systems, layered and mixed networks, transient and distribution analysis, inference, and optimization, we refer the reader to the LINE user manual for this language (`LINE-user-python.pdf`), which expands every topic in this primer in greater depth, and to the LINE example manual (`LINE-examples-python.pdf`), which catalogues the runnable example suite.

Chapter 2

Tutorials

This chapter walks through seventeen complete models, from a single M/M/1 queue to layered, cached, and Petri-net models. Each tutorial states the model, develops the code in reading order, and comments the resulting metrics. The Python source distribution also contains directly runnable versions in `python/examples/gettingstarted`; installed wheels expose the same files as `line_solver.examples.gettingstarted` modules. Those files include the imports and resource handling needed to run them from any working directory. The modelling constructs used here are documented in full in the LINE user manual (`LINE-user-matlab.pdf`, `LINE-user-java.pdf`, `LINE-user-python.pdf`, `LINE-user-cpp.pdf`), and the solution algorithms in the language-specific LINE internals manual (`LINE-internals-python.pdf`).

2.1 Tutorial 1: A M/M/1 queue

The M/M/1 queue is a classic model of a queueing system where jobs arrive into an infinite-capacity buffer, wait to be processed in first-come-first-served (FCFS) order, and then leave after service completion. Arrival and service times are assumed to be independent and exponentially distributed random variables.

In this example, we wish to compute average performance measures for the M/M/1 queue. We assume that arrivals come in at rate $\lambda = 1$ job/s, while service has rate $\mu = 2$ job/s. It is known from theory that the exact value of the server utilization in this case is $\rho = \lambda/\mu = 0.5$, i.e., 50%, while the mean response time for a visit is $R = 1/(\mu - \lambda) = 1$ s. We wish to verify these values using LDES-based discrete-event simulation, instantiated through LINE.

Laid out in the four blocks of Section 1.2, which every script in this chapter marks in its comments, the M/M/1 model is solved as follows

```
from line_solver import *

model = Network('M/M/1')
# Block 1: nodes
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')
# Block 2: classes
jobclass = OpenClass(model, 'Class1')
source.set_arrival(jobclass, Exp(1))
queue.set_service(jobclass, Exp(2))
# Block 3: topology
model.link(Network.serial_routing(source, queue, sink))
# Block 4: solution
avg_table = LDES(model, seed=23000, samples=10000, verbose=VerboseLevel.SILENT).avg_table()
# select a particular table row
print(tget(avg_table, queue, jobclass))
# select a particular table row by node and class label
```

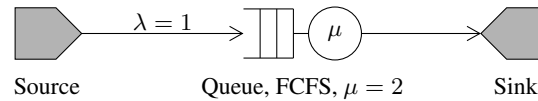


Figure 2.1: The M/M/1 queue of Tutorial 1: a Poisson source feeds a single FCFS station with exponential service, and completed jobs leave through the sink.

```
print(tget(avg_table, 'Queue', 'Class1'))
```

In the example, `source` and `sink` are arrival and departure points of jobs; `queue` is a queueing station with FCFS scheduling; `jobclass` defines an open class of jobs that arrive, get served, and leave the system; `Exp(2.0)` defines an exponential distribution with rate parameter $\lambda = 2.0$; finally, the `avg_table` call solves for average performance measures with LINE's own discrete-event simulator, LDES, using for reproducibility a specific seed for the random number generator.

The result is a table with mean performance measures including: the number of jobs in the station either queueing or receiving service (`QLen`); the utilization of the servers, i.e. the fraction of the station service capacity held by the class (`Util`, see the utilization convention of the LINE user manual); the mean response time for a visit to the station (`RespT`); the mean residence time, i.e. the mean response time cumulatively spent at the station over all visits (`ResidT`); the mean throughput of departing jobs (`Tput`)

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Class1	0	0	0	0	0	1
Queue	Class1	0.9799	0.49595	0.98576	0.98576	0.99486	0.99386

This table is produced by the native `ldes` executable shipped in a source checkout. An installed wheel does not carry that executable and therefore downloads `ldes.jar` on first use. At the same seed the JAR reports the requested arrival rate, `ArvR=1.0`, rather than the native engine's sampled `0.99486`; the remaining figures above are unchanged. A pre-defined gallery of classic models is also available, for example

```
model = gallery_mm1()
```

returns a M/M/1 queue with 50% utilization.

If we want to select a particular row of the `avg_table` data structure, we can use direct object-based indexing or a filtering method. The `tget` function takes either the station and class names or the objects themselves, `tget(avg_table, queue, jobclass)`:

```
ARow = tget(avg_table, 'Queue', 'Class1')
```

gives output

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
1 Queue	Class1	0.9799	0.49595	0.98576	0.98576	0.99486	0.99386

Given a single name, `tget` returns every row that mentions it, so that `tget(avg_table, 'Queue')` filters by station and `tget(avg_table, 'Class1')` by class:

```
CRows = tget(avg_table, 'Class1')
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0 Source	Class1	0.0000	0.00000	0.00000	0.00000	0.00000	1.00000
1 Queue	Class1	0.9799	0.49595	0.98576	0.98576	0.99486	0.99386

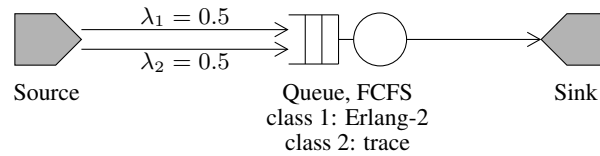


Figure 2.2: The multiclass M/G/1 queue of Tutorial 2: two open classes share one FCFS station, class 1 with Erlang-2 service and class 2 with a trace-driven service time.

2.2 Tutorial 2: A multiclass M/G/1 queue

We now consider a more challenging variant of the first example. We assume that there are two classes of incoming jobs with non-exponential service times. For the first class, service times are Erlang distributed with unit rate and variance $1/3$; they are instead read from a trace for the second class. Both classes have exponentially distributed inter-arrival times with mean $2s$.

The Python listing resolves the bundled trace with `importlib.resources`, so it runs both from a source checkout and from an installed wheel, from any working directory. We start with the node block

```
model = Network('M/G/1')
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')
```

The next step consists in defining the classes. We fit automatically from mean and squared coefficient of variation (i.e., $SCV = \text{variance}/\text{mean}^2$) an Erlang distribution and use the `Replayer` distribution to request that the specified trace is read cyclically to obtain the service times of class 2

```
from importlib.resources import files

jobclass1 = OpenClass(model, 'Class1')
jobclass2 = OpenClass(model, 'Class2')

source.set_arrival(jobclass1, Exp(0.5))
source.set_arrival(jobclass2, Exp(0.5))

queue.set_service(jobclass1, Erlang.fit_mean_and_scv(1, 1/3))
try:
    trace_path = str(files('line_solver.examples.gettingstarted').joinpath('example_trace.txt'))
except ModuleNotFoundError: # unpacked source checkout
    trace_path = lineRootFolder() + '/examples/gettingstarted/example_trace.txt'
queue.set_service(jobclass2, Replayer(trace_path))
```

Note that the `example_trace.txt` file consists of a single column of doubles, each representing a service time value, e.g.,

```
1.2377474e-02
4.4486055e-02
1.0027642e-02
2.0983173e-02
...
```

We now specify a linear route through source, queue, and sink for both classes

```
P = model.init_routing_matrix()
P.set(jobclass1, Network.serial_routing(source, queue, sink))
P.set(jobclass2, Network.serial_routing(source, queue, sink))
model.link(P)
```

and solve the model with LDES

```
ldes_avg_table = LDES(model, seed=23000, samples=10000).avg_table()
print(ldes_avg_table)
```

which gives

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Class1	0	0	0	0	0	0.5
Source	Class2	0	0	0	0	0	0.5
Queue	Class1	0.83114	0.48506	1.7003	1.7003	0.4891	0.48871
Queue	Class2	0.41233	0.050177	0.83801	0.83801	0.49234	0.49165

This is the source-checkout result: MATLAB and Python both drive the native `common/ldes` engine over the same JSON document. An installed Python wheel instead downloads `ldes.jar`; it returns the same queue lengths, utilizations, response times and throughputs, but reports the requested arrival rates 0.5 in both queue rows, as the Java transcript above does. The small Java queue-length difference is within simulation error. We wish now to validate this value against an analytical solver. Since `jobclass2` has trace-based service times, we first need to revise its service time distribution to make it analytically tractable, e.g., we may ask LINE to fit an acyclic phase-type distribution [4] based on the trace

```
queue.set_service(jobclass2, Replayer(trace_path).fit_aph())
```

We can now use a Continuous Time Markov Chain (CTMC) to solve the system, but since the state space is infinite in open models, we need to truncate it to be able to use this solver. For example, we may restrict to states with at most 2 jobs in each class, checking with the `verbose` option the size of the resulting state space

```
ctmc_avg_table2 = CTMC(model, cutoff=2, verbose=True).avg_table()
print(ctmc_avg_table2)
```

which gives

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0 Source	Class1	0.0000	0.0000	0.0000	0.0000	0.0000	0.4411
1 Source	Class2	0.0000	0.0000	0.0000	0.0000	0.0000	0.4758
2 Queue	Class1	0.5674	0.4411	1.2863	1.2863	0.4411	0.4411
3 Queue	Class2	0.2446	0.0481	0.5140	0.5140	0.4758	0.4758

However, we see from the comparison with LDES that the errors of CTMC are rather large. Since the truncated state space consists of just 46 states, we can further increase the cutoff to 4, trading a slower solution time for higher precision

```
ctmc_avg_table4 = CTMC(model, cutoff=4, verbose=True).avg_table()
print(ctmc_avg_table4)
```

which gives

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0 Source	Class1	0.0000	0.0000	0.0000	0.0000	0.0000	0.4916
1 Source	Class2	0.0000	0.0000	0.0000	0.0000	0.0000	0.4957
2 Queue	Class1	0.7958	0.4916	1.6188	1.6188	0.4916	0.4916
3 Queue	Class2	0.3756	0.0501	0.7577	0.7577	0.4957	0.4957

To gain more accuracy, we could either keep increasing the cutoff value or, if we wish to compute an exact solution, we may call the matrix-analytic method (MAM) solver instead. MAM uses the repetitive structure of the CTMC to exactly analyze open systems with an infinite state space, calling

```
print(MAM(model).avg_table())
```

we get

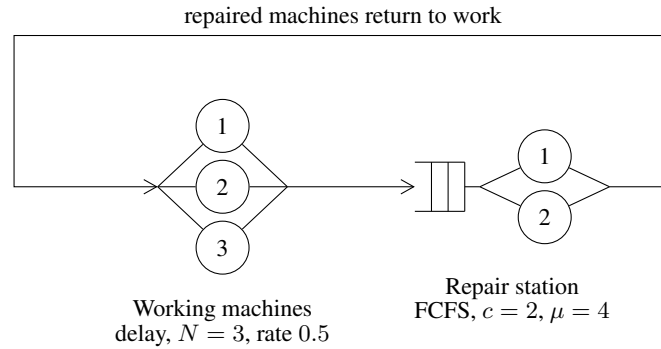


Figure 2.3: The machine interference model of Tutorial 3: $N = 3$ machines alternate between a working state, modelled as a delay, and a two-server FCFS repair station.

	Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0	Source	Class1	0	0	0	0	0.5	0.5
1	Source	Class2	0	0	0	0	0.5	0.5
2	Queue	Class1	0.876460	0.500000	1.752920	1.752920	0.5	0.5
3	Queue	Class2	0.426996	0.050536	0.853991	0.853991	0.5	0.5

The current MAM implementation is primarily constructed on top of Java ports of the BuTools solver [5] and the SMC solver [3].

2.3 Tutorial 3: Machine interference problem

Closed models involve jobs that perpetually cycle within a network of queues. The machine interference problem is a classic example, in which a group of repairmen is tasked with fixing machines as they break and the goal is to choose the optimal size of the group. We here illustrate how to evaluate the performance of a given group size. We consider a scenario with $S = 2$ repairmen, with machines that break down at a rate of 0.5 failed machines/week, after which a machine is fixed in an exponential distributed time with rate 4.0 repaired machines/week. There are a total of $N = 3$ machines.

Suppose that we wish to obtain an exact numerical solution using Continuous Time Markov Chains (CTMCs). The above model can be analyzed as follows:

```
model = Network('MRP')
delay = Delay(model, 'WorkingState')
queue = Queue(model, 'RepairQueue', SchedStrategy.FCFS)
queue.set_number_of_servers(2)
cclass = ClosedClass(model, 'Machines', 3, delay)
delay.set_service(cclass, Exp(0.5))
queue.set_service(cclass, Exp(4.0))
model.link(Network.serialRouting(delay, queue))
solver = CTMC(model)
ctmc_avg_table = solver.avg_table()
print(ctmc_avg_table)
```

Here, `delay` appears in the constructor of the closed class to specify that a job will be considered completed once it returns to the delay (i.e., the machine returns in working state). We say that the delay is thus the *reference station* of `cclass`. The above code prints the following result

	Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0	WorkingState	Machines	2.6648	2.6648	2.0000	2.0000	1.3324	1.3324
1	RepairQueue	Machines	0.3352	0.1666	0.2515	0.2515	1.3324	1.3324

We can now also inspect the CTMC more in the details as follows

```
state_space, node_state_space = solver.state_space()
print(state_space)
```

```
inf_gen, event_filt = solver.generator()
print(inf_gen)
```

which produces in output the state space of the model and the infinitesimal generator of the CTMC

```
[[0. 1. 2.]
 [1. 0. 2.]
 [2. 0. 1.]
 [3. 0. 0.]]

[[-8.  8.  0.  0.]
 [ 0.5 -8.5  8.  0.]
 [ 0.  1. -5.  4.]
 [ 0.  0.  1.5 -1.5]]
```

For example, the first state (0 1 2) consists of two components: the initial 0 denotes the number of jobs in service in the delay, while the remaining part is the state of the FCFS queue. In the latter, the 1 means that a job of class 1 (the only class in this model) is in the waiting buffer, while the 2 means that there are two jobs in service at the queue.

As another example, the second state (1 0 2) is similar, but one job has completed at the queue and has then moved to the delay, concurrently triggering an admission in service for the job that was in the queue buffer. As a result of this, the buffer is now empty. The corresponding transition rate in the infinitesimal generator matrix is row 1 and column 2 of `inf_gen`, which has value 8.0, that is the sum of the completion rates at the queue for each server in the first state, and where indexes 1 and 2 are the rows in `state_space` associated to the source and destination states.

On this and larger infinite generators, we may also list individual non-zero transitions as follows

```
CTMC.print_inf_gen(inf_gen, state_space)
```

gives

```
[ 0.0  1.0  2.0 ] -> [ 1.0  0.0  2.0 ] : 8.0
[ 1.0  0.0  2.0 ] -> [ 0.0  1.0  2.0 ] : 0.5
[ 1.0  0.0  2.0 ] -> [ 2.0  0.0  1.0 ] : 8.0
[ 2.0  0.0  1.0 ] -> [ 1.0  0.0  2.0 ] : 1.0
[ 2.0  0.0  1.0 ] -> [ 3.0  0.0  0.0 ] : 4.0
[ 3.0  0.0  0.0 ] -> [ 2.0  0.0  1.0 ] : 1.5
```

The above printout helps in matching the state transitions to their rates.

To avoid having to inspect the `state_space` variable to determine to which station a particular column refers to, we can alternatively use the more general invocation

```
state_space, node_state_space = solver.state_space()
print(node_state_space)
```

gives

```
[array([[0.],
        [1.],
        [2.],
        [3.]])
 array([[1., 2.],
        [0., 2.],
        [0., 1.],
        [0., 0.]])]
```

which automatically splits the state space into its constituent parts for each stateful node.

A further observation is that `model.state_space()` forces the regeneration of the state space at each invocation, whereas the equivalent function in the CTMC solver, `solver.state_space()`, returns the state space cached during the solution of the CTMC.

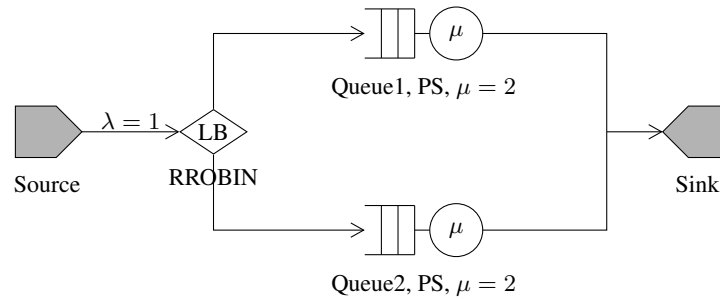


Figure 2.4: The load-balancing model of Tutorial 4: a router dispatches the open stream to two identical processor-sharing queues, first at random and then in round-robin order.

2.4 Tutorial 4: Round-robin load-balancing

In this example we consider a system of two parallel processor-sharing queues and we wish to study the effect of load-balancing on the average performance of an open class of jobs. We begin as usual with the node block, where we now include a special node, called the `Router`, to control the routing of jobs from the source into the queues:

```
model = Network('RRLB')
source = Source(model, 'Source')
lb = Router(model, 'LB')
queue1 = Queue(model, 'Queue1', SchedStrategy.PS)
queue2 = Queue(model, 'Queue2', SchedStrategy.PS)
sink = Sink(model, 'Sink')
```

Let us then define the class block by setting exponentially-distributed inter-arrival times and service times, e.g.,

```
jobclass = OpenClass(model, 'Class1')
source.set_arrival(jobclass, Exp(1.0))
queue1.set_service(jobclass, Exp(2.0))
queue2.set_service(jobclass, Exp(2.0))
```

We now wish to express the fact that the router applies a round-robin strategy to dispatch jobs to the queues. Since this is now a non-probabilistic routing strategy, we need to adopt a slightly different style to declare the routing topology as we cannot specify anymore routing probabilities. First, we indicate the connections between the nodes, using the `add_links` function (in LINE for MATLAB the links are added one at a time, since passing an array of node handles is not supported):

```
model.add_links([[source, lb],
                 [lb, queue1],
                 [lb, queue2],
                 [queue1, sink],
                 [queue2, sink]])
```

At this point, all nodes are automatically configured to route jobs with equal probabilities on the outgoing links (`RoutingStrategy.RAND` policy). If we solve the model at this point, we see that the response time at the queues is around 0.65s.

```
print(LDES(model, seed=23000, samples=10000).avg_table())
```

which gives

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Class1	0	0	0	0	0	1
Queue1	Class1	0.32106	0.24457	0.65206	0.32603	0.49285	0.49236
Queue2	Class1	0.31951	0.24333	0.63701	0.31851	0.50201	0.50151

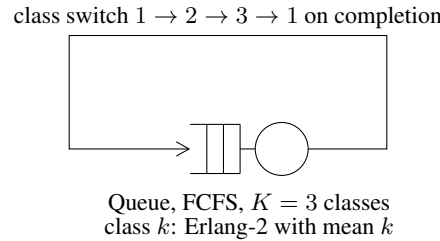


Figure 2.5: The re-entrant line of Tutorial 5: a single FCFS station is visited three times, the class switching on each completion so that every visit draws its service time from a different distribution.

After resetting the internal data structures, which is required before modifying a model we can require LINE to solve again the model using this time a round-robin policy at the router.

```
model.reset()
lb.set_routing(jobclass, RoutingStrategy.RROBIN)
```

Lastly, we run again LDES and find that round-robin produces a visible decrease in response times, which are now around 0.56s.

```
print(LDES(model, seed=23000, samples=10000).avg_table())
```

gives

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Class1	0	0	0	0	0	1
Queue1	Class1	0.28284	0.24697	0.5691	0.28455	0.49749	0.4969
Queue2	Class1	0.27521	0.2408	0.55377	0.27689	0.49739	0.4969

2.5 Tutorial 5: Modelling a re-entrant line

Let us now consider a simple example inspired to the classic problem of modeling *re-entrant lines*. This arises in manufacturing systems where parts (i.e., jobs) re-enter multiple times a machine (i.e., a queueing station), asking at each visit a different class of service. This implies, for example, that the service time at every visit could feature a different mean or a different distribution compared to the previous visits, thus modeling a different stage of processing.

To illustrate this, consider for example a degenerate model composed of a single FCFS queue and K classes. In this model, a job that completes processing in class k is routed back at the tail of the queue in class $k + 1$, unless $k = K$ in which case the job re-enters in class 1.

We take the following assumptions: $K = 3$ and class k has an Erlang-2 service time distribution at the queue with mean equal to k ; the system starts with $N_1 = 1$ jobs in class 1 and zero jobs in all other classes.

```
model = Network('RL')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
K = 3
N = (1, 0, 0)
jobclass = []
for k in range(K):
    jobclass.append(ClosedClass(model, 'Class' + str(k+1), N[k], queue))
    queue.set_service(jobclass[k], Erlang.fit_mean_and_order(1+k, 2))

P = model.init_routing_matrix()
P.set(jobclass[0], jobclass[1], queue, queue, 1.0)
P.set(jobclass[1], jobclass[2], queue, queue, 1.0)
P.set(jobclass[2], jobclass[0], queue, queue, 1.0)
model.link(P)
```

The class-switching rule is enforced automatically, LINE introducing a `ClassSwitch` node in the network when the model is linked.

We can now simulate the performance indexes for the different classes, for example using LINE's normalizing constant solver (NC)

```
ncAvgTable = NC(model).avg_table()
print(ncAvgTable)
```

gives

	Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
0	Queue	Class1	0.166667	0.166667	1.0	0.333333	0.166667	0.166667
1	Queue	Class2	0.333333	0.333333	2.0	0.666667	0.166667	0.166667
2	Queue	Class3	0.500000	0.500000	3.0	1.000000	0.166667	0.166667

Suppose now that the job is considered completed, for the sake of computation of system performance metrics, only when it departs the queue in class K (here `Class3`). By default, LINE will return *system-wide* performance metrics using the `avg_sys_table` method, i.e.,

```
ncAvgSysTable = NC(model).avg_sys_table()
print(ncAvgSysTable)
```

gives

	Chain	JobClasses	SysRespT	SysTput
0	Chain1	(Class1 Class2 Class3)	2.0	0.5

This method identifies the model *chains*, i.e., groups of classes that can exchange jobs with each other, but not with classes in other chains. Since the job can switch into any of the three classes, in this model there is a single chain comprising the three classes.

We see that the throughput of the chain is 0.5, which means that LINE is counting every departure from the queue in any class as a completion for the whole chain. This is incorrect for our model since we want to count completions only when jobs depart in `Class3`. To require this behavior, we can tell to the solver that passages for classes 1 and 2 through the reference station should not be counted as completions

```
jobclass[0].completes = False
jobclass[1].completes = False
```

This modification then gives the correct chain throughput, matching the one of `Class3` alone

```
ncAvgSysTable = NC(model).avg_sys_table()
print(ncAvgSysTable)
```

gives

	Chain	JobClasses	SysRespT	SysTput
0	Chain1	(Class1 Class2 Class3)	6.0	0.166667

In the model above, a job that re-enters the queue in its next class joins the tail of the buffer, so under FCFS it waits behind the other jobs before being served again. Calling `queue.setImmediateFeedback(True)` (or passing a specific class) instead makes such class-switching self-loops *immediate*: the job retains the server and starts its next service without re-queueing, its throughput still being counted and a multi-phase service restarting from phase 1. Immediate feedback is applied by the simulation solvers (LDES and SSA) for the buffered non-preemptive policies (FCFS, HOL, LCFS, SIRO, SEPT, LEPT); it is a no-op at processor-sharing and infinite-server stations (PS, DPS, GPS, INF, LPS), where jobs are never buffered.

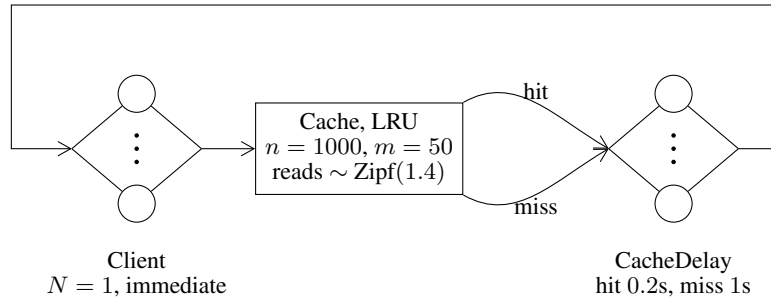


Figure 2.6: The cache-augmented network of Tutorial 6: a single client reads items from an LRU cache, and the read is charged a hit or a miss latency at the cache delay before the client is released.

2.6 Tutorial 6: A queueing network with caching

In this more advanced example, we show how to include in a queueing network a cache adopting a least-recently used (LRU) replacement policy. Under LRU, upon a cache miss the least-recently accessed item will be discarded to make room for the newly requested item.

We consider a cache with a capacity of 50 items, out of a set of 1000 cacheable items. Items are accessed by jobs visiting the cache according to a Zipf-like law with exponent $\alpha = 1.4$ and defined over the finite set of items. A client cyclically issues requests for the items, waiting for a reply before issuing the next request. We assume that a cache hit takes on average $0.2ms$ to process, while a cache miss takes $1ms$. We ask for the average request throughput of the system, differentiated across hits and misses.

Node block As usual, we begin by defining the nodes. Here a delay node will be used to describe the time spent by the requests in the system, while the cache node will determine hits and misses:

```
model = Network('model')
client_delay = Delay(model, 'Client')
cache_node = Cache(model, 'Cache', 1000, 50, ReplacementStrategy.LRU)
cache_delay = Delay(model, 'CacheDelay')
```

Class block We define a set of classes to represent the incoming requests (`client_class`), cache hits (`hit_class`) and cache misses (`miss_class`). These classes need to be closed to ensure that there is a single outstanding request from the client at all times:

```
client_class = ClosedClass(model, 'ClientClass', 1, client_delay, 0)
hit_class = ClosedClass(model, 'HitClass', 0, client_delay, 0)
miss_class = ClosedClass(model, 'MissClass', 0, client_delay, 0)
```

We then assign the processing times, using the `Immediate` distribution to ensure that the client issues immediately the request to the cache:

```
client_delay.set_service(client_class, Immediate())
cache_delay.set_service(hit_class, Exp.fit_mean(0.2))
cache_delay.set_service(miss_class, Exp.fit_mean(1.0))
```

The next step involves specifying that the request uses a Zipf-like distribution (with parameter $\alpha = 1.4$) to select the item to read from the cache, out of a pool of 1000 items

```
cache_node.set_read(client_class, Zipf(1.4, 1000))
```

Finally, we ask that the job should become of class `hit_class` after a cache hit, and should become of class `miss_class` after a cache miss:

```
cache_node.set_hit_class(client_class, hit_class)
cache_node.set_miss_class(client_class, miss_class)
```

Topology block Next, in the topology block we setup the routing so that the request, which starts in `client_class` at the `client_delay`, then moves from there to the cache, remaining in `client_class`

```
P = model.init_routing_matrix()
P.set(client_class, client_class, client_delay, cache_node, 1.0)
```

Internally to the cache, the job will switch its class into either `hit_class` or `miss_class`. Upon departure in one of these classes, we ask it to join in the same class `cache_delay` for further processing

```
P.set(hit_class, hit_class, cache_node, cache_delay, 1.0)
P.set(miss_class, miss_class, cache_node, cache_delay, 1.0)
```

Lastly, the job returns to `client_delay` for completion and start of a new request, which is done by switching its class back to `client_class`

```
P.set(hit_class, client_class, cache_delay, client_delay, 1.0)
P.set(miss_class, client_class, cache_delay, client_delay, 1.0)
```

The above routing strategy is finally applied to the model

```
model.link(P)
```

Solution block To solve the model, since JMT does not support cache modeling, we use the native simulation engine provided within LINE, the SSA solver:

```
ssa_avg_table = SSA(model, samples=20000, seed=1, verbose=True).avg_table()
print(ssa_avg_table)
```

The above script produces the following result

```
SSA analysis [method: serial; type: approximate, randomized; lang: python; env: 3.14.6] completed in 157.331424s.
  Station  JobClass  QLen  Util    RespT  ResidT  ArrR    Tput
0   Client  ClientClass  0.00000  0.00000  1.0000e-08  1.0000e-08  2.9870  2.9713
4 CacheDelay  HitClass  0.49675  0.49675  2.0000e-01  0.16474  2.3460  2.4837
5 CacheDelay  MissClass 0.50325  0.50325  1.0000e+00  0.17628  0.50204 0.50325
```

The departing flows from the `cache_delay` are the hit and miss rates. Read them from the corresponding `Tput` rows; they vary slightly with the simulator and random stream.

We can also run the same 20,000-sample budget as independent seeded replicas with the parallel method of SSA

```
ssa_avg_table_para = SSA(model, method='parallel', samples=20000, seed=1).avg_table()
print(ssa_avg_table_para)
```

This gives us a rather similar result, when run on a dual-core machine

```
print(ssa_avg_table_para)
```

The execution time is longer than usual at the first invocation of the parallel solver due to the time needed by MATLAB to bootstrap the parallel pool, in this example around 22 seconds. Successive invocations of parallel SSA normally take much less, with this example around 7 seconds each.

Cache-specific result tables. Besides the standard per-station table, LINE reports two cache-specific tables. The `get_avg_cache_table` method summarizes the hit and miss probabilities for each

cache node and requesting class, while `get_avg_item_table` reports the per-item occupancy, i.e. the steady-state probability that each item resides in each cache list. These tables are produced by the analytical solvers; here we use MVA:

```
solver = MVA(model)
print(solver.get_avg_cache_table())
print(solver.get_avg_item_table())
```

The cache table reports an overall hit probability of about 0.771 for the requesting class

Node	JobClass	List	ListCap	Items	HitProb	MissProb
Cache	ClientClass	0	50	1000	0.77121	0.22879

The item table contains one row per (item, list); with an LRU policy the most popular items are almost always resident, and the per-item occupancies sum to the cache capacity (here 50):

Node	Item	List	ListCap	Prob
Cache	1	1	50	1.00000
Cache	2	1	50	1.00000
Cache	3	1	50	1.00000
Cache	4	1	50	0.99987
...	...	...	...	...

The per-item table is reported from the scalable TTL algorithm for LRU caches; for RR/FIFO caches it relies on the exact product-form recursion and is therefore reported only when the cache holds at most ten items (otherwise a warning is issued and the table reports NaN).

2.7 Tutorial 7: Delayed-hit caches with a retrieval system

In a *delayed-hit* cache a miss does not complete instantaneously: the requested item must be fetched through a *retrieval system*, a small network of queues that models the cost of retrieving the item (e.g. a back-end server or a network channel). While item i is being fetched, further requests for the same item become *delayed hits*: they wait for the in-flight fetch to complete rather than triggering a new retrieval. LINE represents the retrieval system with `Cache.setRetrievalSystem` and analyses it exactly with NC (a product-form recurrence) and approximately, including the expected latency, with MVA (a fixed-point heuristic).

We reproduce a motivating example with $n = 7$ items, a single cache list of capacity $m = 6$ under random-replacement (RR), and a single processor-sharing (PS) retrieval station of unit service rate. Requests arrive in a Poisson stream and read items with the popularity vector $\lambda = (49, 49, 49, 49, 7, 1, 1)/205$.

Model On a miss, the read request switches to a per-item retrieval class that is routed through the retrieval queue and back to the cache. `setRetrievalSystem` declares the retrieval classes and stations; the routing matrix must still contain the cache→queue→cache path. The retrieval service time is taken from the read class's service distribution at each queue, set with `set_service`.

```
import numpy as np

access_prob = list(np.array([49, 49, 49, 49, 7, 1, 1]) / 205) # item popularities
service_rates = [1.0] * 7 # per-item fetch rates
n = len(service_rates)

model = Network('Retrieval')
source = Source(model, 'Source')
cache_node = Cache(model, 'Cache', n, [6], ReplacementStrategy.RR)
queue = Queue(model, 'Queue', SchedStrategy.PS) # retrieval station
sink = Sink(model, 'Sink')

job_class = OpenClass(model, 'InitClass', 0)
hit_class = OpenClass(model, 'HitClass', 0)
```

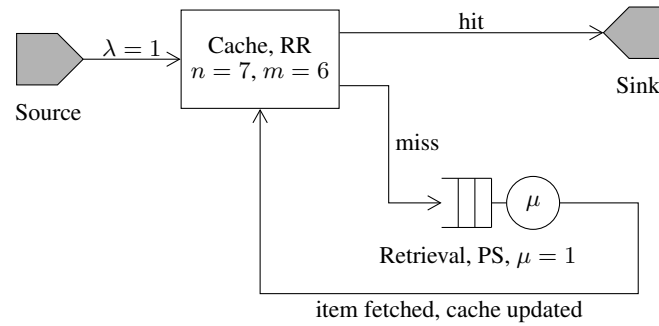


Figure 2.7: The delayed-hit cache of Tutorial 7 where a miss is routed to a processor-sharing retrieval station, and only when the fetch completes is the cache updated and the request released.

```
miss_class = OpenClass(model, 'MissClass', 0)

source.set_arrival(job_class, Exp(1))
cache_node.set_read(job_class, DiscreteSampler(access_prob))
cache_node.set_hit_class(job_class, hit_class)
cache_node.set_miss_class(job_class, miss_class)
queue.set_service(job_class, Exp(1)) # retrieval service is taken from the read class
cache_node.set_retrieval_system(job_class, miss_class, queue)

P = model.init_routing_matrix()
P.set(job_class, job_class, source, cache_node, 1.0)
P.set(job_class, job_class, cache_node, queue, 1.0)
P.set(job_class, job_class, queue, cache_node, 1.0)
P.set(hit_class, hit_class, cache_node, sink, 1.0)
P.set(miss_class, miss_class, cache_node, sink, 1.0)
model.link(P)
```

Exact analysis NC detects the retrieval system and applies the exact product-form recurrence, returning the hit and miss probabilities of the read class.

```
NC(model).get_avg_table()
hit = cache_node.get_hit_ratio() # 0.982722 (exact)
miss = cache_node.get_miss_ratio() # 0.015118
```

The cache getters return fixed-width NumPy arrays. In this one-read-class model the first entry is the value shown in the comments; unused entries are NaN padding.

Approximate analysis and latency MVA uses the fixed-point heuristic to obtain the expected retrieval latency Z , the mean delay incurred by misses and delayed hits. This latency is reported in the `ResidT` column of `getAvgCacheTable` and is populated by the MVA, NC, CTMC, SSA and LDES solvers: the analytical solvers obtain it from the retrieval recurrence, the simulators measure it directly on the sample path. Solvers that do not model the retrieval sub-network leave the column unset (NaN).

```
MVA(model).get_avg_table()
hit = cache_node.get_hit_ratio() # 0.975727 (FPI)
residt = cache_node.get_residt() # 1.018005
```

As above, read the first entry for this one-read-class model; the remaining array entries are unused NaN padding.

Retrieval station types The retrieval system may contain infinite-server (IS) stations and single-server stations with PS, SIRO, FCFS or LCFS-PR scheduling. PS and LCFS-PR (symmetric, insensitive disciplines) admit general phase-type service and class-dependent rates; SIRO and FCFS require exponential

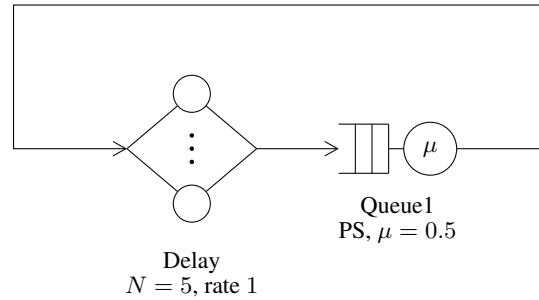


Figure 2.8: The closed network of Tutorial 8, used to compare the response time distribution of the processor-sharing queue obtained by fluid analysis and by simulation.

service with identical per-class service rates. The discrete-event simulator LDES also simulates delayed-hit caches and measures the retrieval latency directly, providing a cross-check for the analytical solvers.

2.8 Tutorial 8: Response time distribution and percentiles

In this example we illustrate the computation of response time percentiles in a queueing network model. We begin by instantiating a simple closed model consisting of a delay followed by a processor-sharing (PS) queueing station.

```
import numpy as np

model = Network('Model')

node = np.empty(2, dtype=object)
node[0] = Delay(model, 'Delay')
node[1] = Queue(model, 'Queue1', SchedStrategy.PS)
```

There is a single class consisting of 5 jobs that circulate between the two stations, taking exponential service times at both.

```
jobclass = np.empty(2, dtype=object)
jobclass[0] = ClosedClass(model, 'Class1', 5, node[0], 0)
node[0].set_service(jobclass[0], Exp(1.0))
node[1].set_service(jobclass[0], Exp(0.5))

model.link(Network.serial_routing(node[0], node[1]))
```

We now wish to compare the response time distribution at the PS queue computed analytically with a fluid approximation against the simulated values returned by LDES. To do so, we call the `get_cdf_resp_t` method

```
rd_fluid = FLD(model).get_cdf_resp_t()
```

```
# rd_fluid = FLD(model).get_cdf_resp_t()
rd_sim = LDES(model, seed=23000, samples=10000).get_cdf_resp_t()
```

The returned data structures, `rd_fluid` and `rd_sim`, are nested lists where `[i][r]` is the two-column CDF array for station i and class r . The first column represents the cumulative distribution function (CDF) value $F(t) = Pr(T \leq t)$, where T is the random variable denoting the response time, while t is the percentile appearing in the corresponding entry of the second column.

For example, to plot the complementary CDF $1 - F(t)$ we can use the following code

```
# Plotting in Python using matplotlib
import matplotlib.pyplot as plt
```

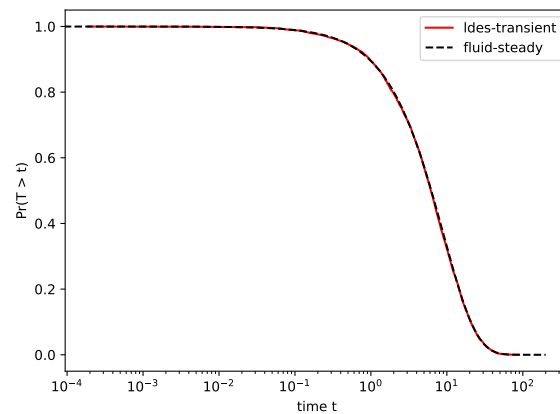


Figure 2.9: Comparison of simulated response time distribution and its fluid approximation

```
plt.semilogx(rd_sim[1][0][:, 1], 1 - rd_sim[1][0][:, 0], 'r', label='ldes-transient')
plt.semilogx(rd_fluid[1][0][:, 1], 1 - rd_fluid[1][0][:, 0], '--', label='fluid-steady')
plt.legend()
plt.ylabel('Pr(T > t)')
plt.xlabel('time t')
plt.show()
```

which produces the graph shown in Figure 2.9. The graph shows that, although the simulation refers to a transient, while the fluid approximation refers to steady-state, there is a tight matching between the two response time distributions.

We can readily compute the percentiles from the `rd_fluid` and `rd_sim` data structures, e.g., for the 95th and 99th percentiles of the simulated distribution

```
# Calculate 95th and 99th percentiles from CDF data
cdf_data = rd_sim[1][0] # station 2, class 1
mask_95 = cdf_data[:, 0] < 0.95
prc95 = np.max(cdf_data[mask_95, 1]) if np.any(mask_95) else 0
mask_99 = cdf_data[:, 0] < 0.99
prc99 = np.max(cdf_data[mask_99, 1]) if np.any(mask_99) else 0
```

That is, 95% of the response times at the PS queue (node 2, class 1) are less than or equal to 26.5369 time units, while 99% are less than or equal to 41.6524 time units.

2.9 Tutorial 9: Optimizing a performance metric

In this example, we show how to optimize with the help of LINE a performance metric. We wish to find the optimal routing probabilities that minimize average response times for two parallel processor sharing queues. We assume that jobs are fed by a delay station, arranged with the two queues in a closed network topology.

In Python the complete objective function is defined after the model-building statements below, under the name `routing_model`. Within the function definition, we instantiate the two queues and the delay station

```
model = Network('LoadBalCQN')
delay = Delay(model, 'Think')
queue1 = Queue(model, 'Queue1', SchedStrategy.PS)
queue2 = Queue(model, 'Queue2', SchedStrategy.PS)
```

We assume that 16 jobs circulate among the nodes, and that the service rates are $\sigma = 1$ jobs per unit time at the delay, and $\mu_1 = 0.75$ and $\mu_2 = 0.50$ at the two queues:

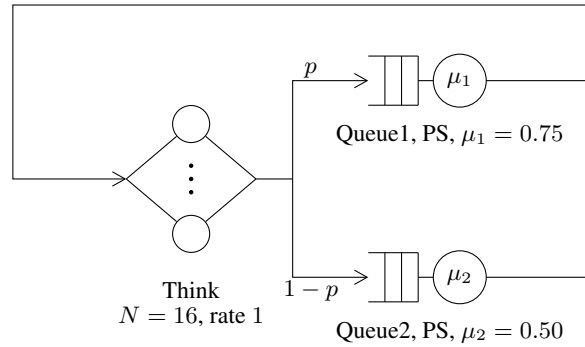


Figure 2.10: The load-balancing closed network of Tutorial 9: the routing probability p that splits the think-time output between the two queues is the decision variable of the optimization.

```

cclass = ClosedClass(model, 'Job1', 16, delay)
delay.set_service(cclass, Exp(1))
queue1.set_service(cclass, Exp(0.75))
queue2.set_service(cclass, Exp(0.50))

```

We initially setup a topology with arbitrary values for the routing probabilities between delay and queues, ensuring that jobs completing at the queues return to the delay:

```

P = model.init_routing_matrix()
P.set(cclass, cclass, queue1, delay, 1.0)
P.set(cclass, cclass, queue2, delay, 1.0)
model.link(P)

```

We now return the system response time for the jobs as a function of the routing probability p to choose queue 1 instead of queue 2: Lastly, we optimize the function we defined

```

def routing_model(p):
    # Block 4: solution
    P.set(cclass, cclass, delay, queue1, p)
    P.set(cclass, cclass, delay, queue2, 1.0 - p)
    model.reset()
    model.link(P)
    solver = MVA(model, method='exact')
    return solver.avg_sys_resp_t()[0]

```

Lastly, we optimize the function we defined

```

from scipy import optimize
p_opt = optimize.fminbound(routing_model, 0, 1)
print(f'Optimal p: {p_opt}')

```

We are now ready to run the example. The execution returns the optimal value 0.6104878504366782.

2.10 Tutorial 10: Studying a departure process

This examples illustrates LINE's support for extracting simulation data about particular events in an extended queueing network, such as departures from a particular queue.

Our goal is to obtain the squared coefficient of variation of the inter-departure times from a $M/E_2/1$ queue, which has Poisson arrivals and 2-phase Erlang distributed service times.

Because this is a classic model, we can find it in LINE's model gallery. The additional return parameters (e.g., `source, queue, ...`) provide handles to the entities within the model.

```

import numpy as np

```

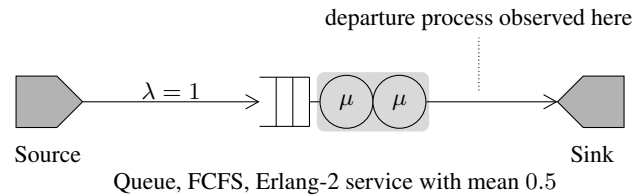


Figure 2.11: The $M/E_2/1$ queue of Tutorial 10: the two shaded circles are the phases of the Erlang-2 service time, and the departure process is observed at the station output.

```
model, source, queue, sink, jobclass = gallery_mer11()
```

We now simulate 100,000 transition events from the underpinning continuous-time Markov chain

```
solver = CTMC(model, cutoff=150, seed=23000)
sa = solver.sample_sys_aggr(100000)
```

The returned data structure supplies the state trajectory of the stateful nodes (here `source` and `queue`), its timestamps, and the 100,000 transition events. The number of stored state timestamps is solver-specific because a transition can add more than one state record.

```
# Display structure of sampled aggregated data
print('Sample aggregate data structure:')
print(f'State data points: {len(sa.t)}')
print(f'Event data available: {len(sa.event) > 0}')
```

Each event records its timestamp, node, class and event type. The following code inspects the first two records; their exact order and timestamps depend on the language implementation's event scheduling convention.

```
# Access event data (sa.event is a list of EventInfo objects)
event1 = sa.event[0]
print(f'Event 1 - Node: {event1.node}, Event: {event1.event}')
event2 = sa.event[1]
print(f'Event 2 - Node: {event2.node}, Event: {event2.event}')
```

We may also plot the first 300 events as follows

```
# Plot first 300 events sample path
import matplotlib.pyplot as plt
queue_index = model.get_node_index(queue) - 1
plt.plot(sa.t[:300], sa.state[queue_index][:300])
plt.xlabel('Time')
plt.ylabel('Queue Length')
plt.title('Sample Path for M/E2/1 Queue')
plt.show()
```

We are now ready to filter the timestamps of events related to departures from the `queue` node

```
# Filter events for departures from queue node
queue_index = model.get_node_index(queue) - 1
filt_event = [e.node == queue_index and e.event == 'DEP' for e in sa.event]
```

Followed by a calculation of the time series of inter-departure times, whose leading entries are

```
# Calculate inter-departure times (continued from above)
departure_times = [event.t for event, is_dep in zip(sa.event, filt_event) if is_dep]
inter_dep_times = np.diff(departure_times)
print(f'Leading inter-departure times: {inter_dep_times[:5]}')
```

We may now for example compute the squared coefficient of variation of this process

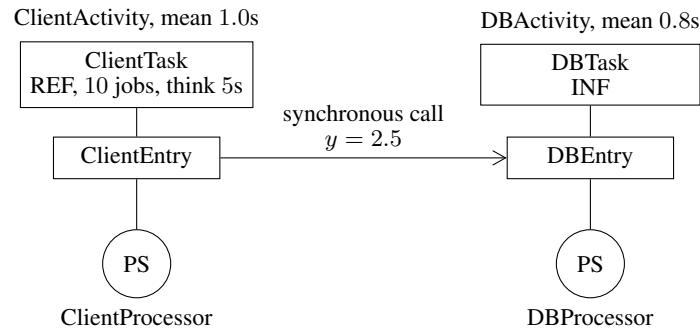


Figure 2.12: The layered network of Tutorial 11: the client task issues on average 2.5 synchronous calls per request to the database entry, and each task runs on its own processor.

```
# Calculate squared coefficient of variation
scv_d_est = np.var(inter_dep_times) / np.mean(inter_dep_times)**2
print(f'SCV estimate: {scv_d_est}')
```

With the seed and 100,000 events above, this run evaluates to approximately 0.8778. Using Marshall's exact formula for the $GI/G/1$ queue [7], we get a theoretical value of 0.8750, in close agreement with the simulation estimate.

2.11 Tutorial 11: Basic layered queueing network

This example demonstrates how to model and analyze a basic layered queueing network (LQN) representing a simple client-server application with two tiers: a client layer and a database layer. The LQN consists of a client processor P1 with a reference task T1 (10 users), a database processor P2 with an infinite server task T2, synchronous calls from the client to the database, exponential service times at both layers.

We start by creating the layered network instance:

```
model = LayeredNetwork('ClientDBSystem')
```

Next, we define the processors and tasks:

```
# Create processors
P1 = Processor(model, 'ClientProcessor', 1, SchedStrategy.PS)
P2 = Processor(model, 'DBProcessor', 1, SchedStrategy.PS)

# Create tasks
T1 = Task(model, 'ClientTask', 10, SchedStrategy.REF).on(P1)
T1.set_think_time(Exp.fit_mean(5.0)) # 5-second think time
T2 = Task(model, 'DBTask', float('inf'), SchedStrategy.INF).on(P2)
```

Now we define the entries that represent service interfaces:

```
E1 = Entry(model, 'ClientEntry').on(T1)
E2 = Entry(model, 'DBEntry').on(T2)
```

Finally, we define the activities that specify the service demand and the synchronous calls

```
# Client activity: processes request and calls DB
A1 = Activity(model, 'ClientActivity', Exp.fit_mean(1.0)).on(T1)
A1.bound_to(E1).synch_call(E2, 2.5) # 2.5 DB calls on average

# DB activity: processes database request
A2 = Activity(model, 'DBActivity', Exp.fit_mean(0.8)).on(T2)
A2.bound_to(E2).replies_to(E2)
```

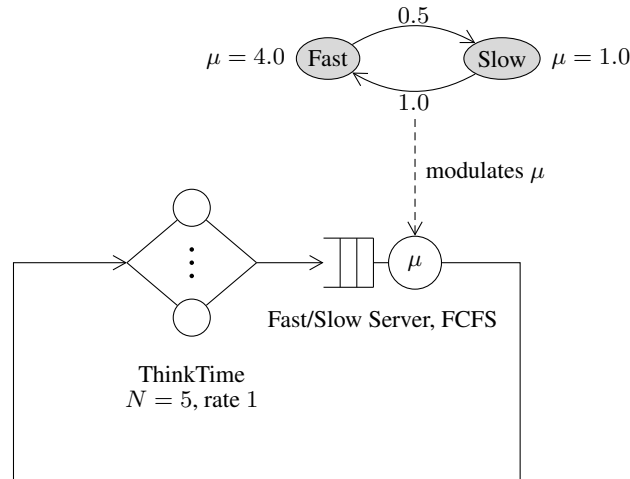


Figure 2.13: The random environment of Tutorial 12: a two-state Markov chain alternates between a fast and a slow mode, modulating the service rate of the queue in the underlying closed network.

Now we solve the layered network using the LN solver with MVA applied to each layer:

```
solver = LN(model, lambda m: MVA(m))
avg_table = solver.avg_table()
print(avg_table)
```

The output shows the performance metrics for each node in the layered network:

Node	NodeType	QLen	Util	RespT	ResidT	ArvR	Tput
ClientProcessor	Processor	NaN	0.49912	NaN	NaN	NaN	NaN
DBProcessor	Processor	NaN	0.99823	NaN	NaN	NaN	NaN
ClientTask	RefTask	7.5	0.49912	NaN	1.7253	NaN	0.49912
DBTask	Task	6.6433	0.99823	NaN	5.324	NaN	1.2478
ClientEntry	Entry	7.5	0.49912	15.035	NaN	NaN	0.49912
DBEntry	Entry	6.6433	0.99823	5.324	NaN	NaN	1.2478
ClientActivity	Activity	7.5	0.49912	15.035	1.7253	NaN	0.49912
DBActivity	Activity	6.6433	0.99823	5.324	5.324	NaN	1.2478

2.12 Tutorial 12: Random environments

This tutorial illustrates how to model a queueing system operating in a random environment, in which the parameters of the network depend on the state of an underlying continuous-time Markov chain. We consider a simple closed network with one delay station and one FCFS queue, and we let the queue service rate depend on a binary environment that alternates between a `Fast` mode (service rate $\mu = 4.0$) and a `Slow` mode ($\mu = 1.0$). The environment leaves `Fast` at rate 0.5 and `Slow` at rate 1.0, so the stationary distribution is $\Pr(\text{Fast}) = 2/3$, $\Pr(\text{Slow}) = 1/3$. A complete description of the random-environment formalism and of the ENV solver is given in the chapter on random environments of the LINE user manual; here we focus on the construction pattern.

We start by building a base network with a placeholder service rate at the queue:

```
base_model = Network('BaseModel')
delay = Delay(base_model, 'ThinkTime')
queue = Queue(base_model, 'Fast/Slow Server', SchedStrategy.FCFS)
N = 5
jobclass = ClosedClass(base_model, 'Jobs', N, delay)
delay.setService(jobclass, Exp(1.0))
queue.setService(jobclass, Exp(2.0))
base_model.link(Network.serialRouting(delay, queue))
```

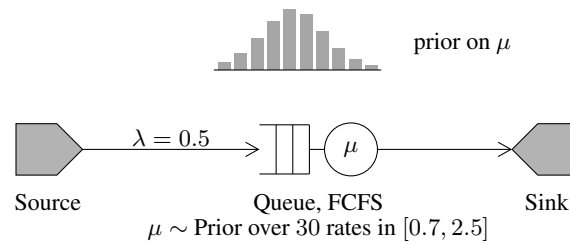


Figure 2.14: The model of Tutorial 13: an M/M/1 queue whose service rate is not known exactly but is given by a prior over 30 alternatives.

We then build the environment by adding two stages, each holding an independent copy of the base model whose queue service rate has been overridden:

```
env = Environment('ServerModes', 2)

fast_model = base_model.copy()
fast_model.getNodeByName('Fast/Slow Server').setService(fast_model.classes[0], Exp(4.0))
env.addStage(0, 'Fast', 'operational', fast_model)

slow_model = base_model.copy()
slow_model.getNodeByName('Fast/Slow Server').setService(slow_model.classes[0], Exp(1.0))
env.addStage(1, 'Slow', 'degraded', slow_model)

env.addTransition(0, 1, Exp(0.5))
env.addTransition(1, 0, Exp(1.0))
```

The environment is solved by the ENV meta-solver, which takes a factory that produces an inner solver for each stage. We use the FLD (fluid) solver in each stage of the environment:

```
env_solver = ENV(env, lambda m: FLD(m))
print(env_solver.getAvgTable())
```

The returned table reports performance metrics averaged over the stationary distribution of the environment. The two stages can also be analyzed independently in steady-state with MVA, recovering the conditional metrics for each mode. See the chapter on random environments of the LINE user manual for breakdown/repair models, semi-Markovian transitions, and other refinements.

2.13 Tutorial 13: Posterior analysis

This tutorial demonstrates how to propagate parameter uncertainty through a queueing model using the UQ solver. The idea is to attach a `Prior` distribution to a model parameter, then ask the solver to evaluate the network for each alternative and aggregate the resulting performance metrics into a posterior distribution.

We consider an M/M/1 queue with arrival rate $\lambda = 0.5$ and an uncertain service rate. The uncertainty is modelled by a `Prior` with 30 alternatives drawn uniformly from $[0.7, 2.5]$ and weighted by a Gaussian-shaped prior centred at $\mu = 1.3$ with standard deviation 0.4:

```
import numpy as np
model = Network('UncertainServiceModel')
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')
jobClass = OpenClass(model, 'Jobs')
source.setArrival(jobClass, Exp(0.5))

num_alternatives = 30
service_rates = np.linspace(0.7, 2.5, num_alternatives)
```

```
prior_mean, prior_std = 1.3, 0.4
prior_probs = np.exp(-0.5 * ((service_rates - prior_mean) / prior_std) ** 2)
prior_probs = prior_probs / np.sum(prior_probs)

alternatives = [Exp(rate) for rate in service_rates]
queue.setService(jobClass, Prior(alternatives, prior_probs.tolist()))

model.link(Network.serial_routing([source, queue, sink]))
```

The model is solved by wrapping a back-end solver (here MVA) inside the UQ meta-solver. UQ runs the back-end once per alternative and aggregates the results:

```
post = UQ(model, MVA)
post.run_analyzer()

avg_table = post.get_avg_table() # prior-weighted means
post_table = post.get_posterior_table() # per-alternative results
resp_dist = post.get_posterior_dist('R', queue, jobClass)
qlen_dist = post.get_posterior_dist('Q', queue, jobClass)
util_dist = post.get_posterior_dist('U', queue, jobClass)
```

The objects returned by `get_posterior_dist` are empirical CDFs over the requested metric. Their `data` field is an $n \times 2$ matrix whose first column is the cumulative probability and whose second column is the metric value, from which one obtains expectations, modes, and quantiles in the usual way:

```
resp_values = resp_dist.data[:, 1]
resp_cdf = resp_dist.data[:, 0]
resp_probs = np.concatenate([[resp_cdf[0]], np.diff(resp_cdf)])
expected_R = np.sum(resp_values * resp_probs)
median_R = resp_values[np.searchsorted(resp_cdf, 0.5)]
```

It is often more informative to visualise the whole posterior distribution rather than a single summary. Differencing the empirical CDF yields the posterior probability masses, which we normalise by the bin widths to obtain a density estimate; plotting this posterior PDF shows how the parameter uncertainty spreads the response time over a range of values:

```
import matplotlib.pyplot as plt
edges = np.concatenate([[resp_values[0]],
                        (resp_values[:-1] + resp_values[1:]) / 2,
                        [resp_values[-1]]])
widths = np.diff(edges)
density = resp_probs / widths
plt.bar(resp_values, density, width=widths, align='center')
plt.xlabel('response time R'); plt.ylabel('posterior PDF f(R)')
plt.axvline(expected_R, linestyle='--', color='k')
plt.axvline(median_R, linestyle='--', color='r')
plt.show()
```

The resulting density (Figure 2.15) is right-skewed: slow-service alternatives in the prior contribute a heavy upper tail, so the posterior mean of R lies well above its median. This pattern lets one quantify, for any metric of interest, both the central tendency and the dispersion induced by parameter uncertainty without re-implementing the analytical solver loop.

2.14 Tutorial 14: Open cluster

In this tutorial we model an open cluster: a single open class of jobs flows from a `Source` through a `Router` (the dispatcher) into one of $M = 3$ parallel processor-sharing servers, after which it leaves the system through a `Sink`. We illustrate two equivalent ways of building this model: a one-liner factory `Network.cluster_ps`, and the more flexible `line_solver.gen.Cluster` builder, which we use to compare two dispatching policies on the same cluster.

We start with the one-liner factory. The arrival rate is $\lambda = 0.4$ and each server has a mean service

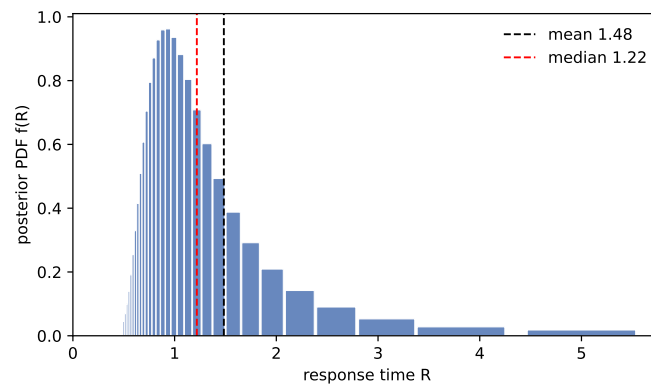


Figure 2.15: Posterior PDF of the response time R for the $M/M/1$ queue with an uncertain service rate. The distribution is right-skewed: the posterior mean (≈ 1.48) exceeds the median (≈ 1.22) because the low-service-rate alternatives contribute a long upper tail.

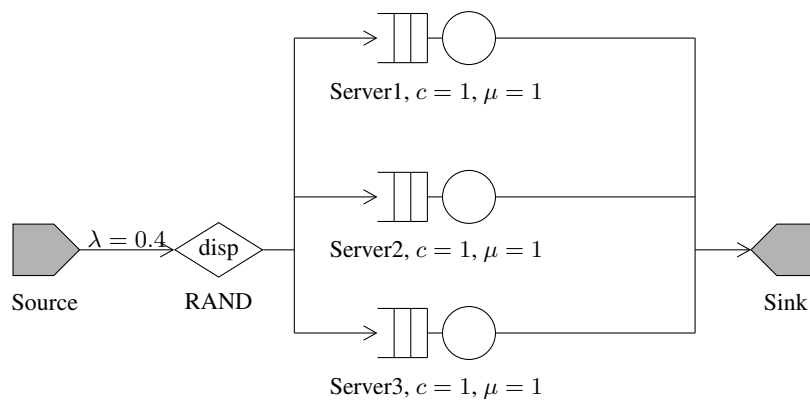


Figure 2.16: The initial open cluster of Tutorial 14: an arrival stream is dispatched at random over three single-server stations.

time of $D = 1$, giving a per-server utilization of $\lambda/M \approx 0.133$ under random dispatching:

```
lam = [0.4]
D = [[1.0], [1.0], [1.0]]
model = Network.cluster_ps(lam, D, RoutingStrategy.RAND)
print(MVA(model).get_avg_table())
```

The MVA result shows traffic split evenly across the three servers, as expected under random dispatching:

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Class1	0	0	0	0	0	0.4
Server1	Class1	0.15385	0.13333	1.1538	0.38462	0.13333	0.13333
Server2	Class1	0.15385	0.13333	1.1538	0.38462	0.13333	0.13333
Server3	Class1	0.15385	0.13333	1.1538	0.38462	0.13333	0.13333

We now switch to the Cluster builder to introduce non-uniform multi-server queues, replacing PS with FCFS and giving Server1 two parallel cores:

```
cluster = (Cluster().set_num_stations(3).set_arrival_rate(0.4).set_service_rate(1.0)
            .set_scheduling(SchedStrategy.FCFS)
            .set_station_servers([2, 1, 1]))
print(MVA(cluster.build()).get_avg_table())
```

We then cross-check the same FCFS cluster under two simulators. LDES is LINE's discrete-event simulator built on the SSJ library, invoked as a subprocess; SSA is LINE's native next-reaction-method stochastic simulator. Both produce statistically equivalent results on this open-class cluster and exercise the multi-server FCFS path.

```
print(LDES(cluster.build(), seed=23000, samples=20000).get_avg_table())
print(SSA(cluster.build(), seed=23000, samples=20000).get_avg_table())
```

The two tables agree to within simulation error. The Python transcript below comes from the native `ldes` executable in the source checkout; an installed wheel falls back to `ldes.jar` and prints the Java-edition LDES figures immediately above instead:

```
=== LDES ===
Station  JobClass      QLen      Util      RespT      ResidT      ArvR      Tput
Source   Class1          0          0          0          0          0          0.4
Station1 Class1      0.13188  0.065715  0.99936  0.33312  0.13209  0.13195
Station2 Class1      0.14731  0.12868  1.1232  0.37441  0.1313  0.13116
Station3 Class1      0.15404  0.13227  1.1503  0.38343  0.134  0.13388
=== SSA ===
Station  JobClass      QLen      Util      RespT      ResidT      ArvR      Tput
Source   Class1          0          0          0          0          0          0.4
Station1 Class1      0.13578  0.067733  1.0023  0.33412  0.13333  0.13547
Station2 Class1      0.14983  0.12953  1.1567  0.38556  0.13333  0.12953
Station3 Class1      0.15859  0.14016  1.1315  0.37716  0.13333  0.14016
```

Finally, we compare random and round-robin dispatching on a uniform PS cluster. Round-robin is non-product-form, so we drop to LDES with a small sample budget:

```
cluster2 = (Cluster().set_num_stations(3).set_arrival_rate(0.4).set_service_rate(1.0)
            .set_scheduling(SchedStrategy.PS))
solver_fcn = lambda m: LDES(m, seed=23000, samples=5000).get_avg_table()
for policy in (RoutingStrategy.RAND, RoutingStrategy.RROBIN):
    cluster2.set_dispatching(policy)
    print(solver_fcn(cluster2.build()))
```

```
=== Dispatching: 0 ===
Station  JobClass      QLen      Util      RespT      ResidT      ArvR      Tput
Source   Class1          0          0          0          0          0          0.4
Station1 Class1      0.14723  0.13065  1.1276  0.37587  0.1306  0.13052
Station2 Class1      0.13899  0.12193  1.0954  0.36515  0.12684  0.12661
Station3 Class1      0.14925  0.13166  1.1118  0.37058  0.13436  0.13421
=== Dispatching: 2 ===
Station  JobClass      QLen      Util      RespT      ResidT      ArvR      Tput
Source   Class1          0          0          0          0          0          0.4
Station1 Class1      0.13374  0.13062  1.0246  0.34153  0.13062  0.13046
Station2 Class1      0.12748  0.12581  0.97613  0.32538  0.13062  0.13046
Station3 Class1      0.13118  0.12817  1.0049  0.33498  0.13054  0.13046
```

As in the first comparison, the installed-wheel fallback prints the Java-edition LDES figures; both engines agree with SSA within simulation error. For this lightly-loaded cluster both policies yield very similar response times; differences become visible only at higher utilization, where round-robin reduces variability across servers compared with the i.i.d. split produced by random dispatching.

2.15 Tutorial 15: A loss network

A *loss network* models a system in which an arriving job (a *call* or *connection*) is admitted only if enough resources are free; otherwise it is *blocked* and lost rather than queued [6]. Such models arise in circuit-switched telephony and in multirate broadband networks, where each class of call requires a fixed number of circuits on each link along its route.

In LINE a loss network is expressed as an open model whose stations sit inside a *finite capacity region* (FCR) with the DROP policy: when the region is full, arriving jobs are dropped. The stationary distribution is of product form, and the quantities of interest are the per-class blocking probabilities and

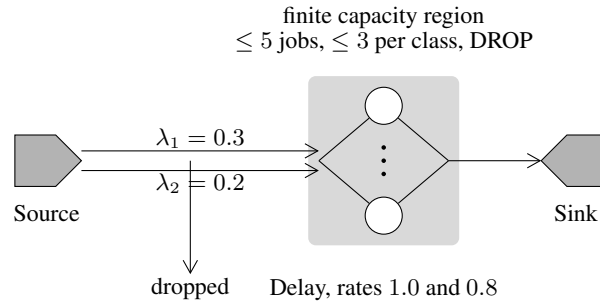


Figure 2.17: The loss network of Tutorial 15: the delay station belongs to a finite capacity region, and arrivals that would violate the global or the per-class limit are dropped.

the normalization constant, the sum being taken over all admissible states Ω .

In this example two classes of calls share an infinite-server Delay station. Class 1 offers load $\rho_1 = 0.3$ and class 2 offers $\rho_2 = 0.2$ (arrival rate over service rate). The region admits at most 5 calls in total and at most 3 of each class:

```
model = Network('LossNetwork')
# Block 1: nodes
source = Source(model, 'Source')
delay = Delay(model, 'Delay')
sink = Sink(model, 'Sink')
# Block 2: classes
class1 = OpenClass(model, 'Class1')
class2 = OpenClass(model, 'Class2')
source.set_arrival(class1, Exp(0.3))
source.set_arrival(class2, Exp(0.2))
delay.set_service(class1, Exp(1.0))
delay.set_service(class2, Exp(0.8))
# Block 3: topology
P = model.init_routing_matrix()
P.set(class1, class1, source, delay, 1.0)
P.set(class1, class1, delay, sink, 1.0)
P.set(class2, class2, source, delay, 1.0)
P.set(class2, class2, delay, sink, 1.0)
model.link(P)
# Block 4: finite capacity region with dropping
fcr = model.add_region(delay)
fcr.set_global_max_jobs(5)
fcr.set_class_max_jobs(class1, 3)
fcr.set_class_max_jobs(class2, 3)
fcr.set_drop_rule(class1, True)
fcr.set_drop_rule(class2, True)
# Block 5: solution
# Exact loss-network transform (the default)
print(SolverNC(model).avg_table())
# Monte Carlo summation (Ross-Wang): also estimates the normalizing constant
print(SolverNC(model, method='mci', samples=100000, seed=23000).avg_table())
# Simulation cross-check: LDES simulates the DROP region directly
print(SolverLDES(model, seed=23000, samples=100000).avg_table())
```

The three solutions agree closely, confirming the product-form result against simulation:

AvgTable =		% SolverNC, lossn.exact (default)		Util		RespT		ResidT		ArvR		Tput	
Station	JobClass	QLen											
Delay	Class1	0.29898	0.29898	1	1	0.29898	0.29898						
Delay	Class2	0.24947	0.24947	1.25	1.25	0.19958	0.19958						
AvgTableMCI =		% SolverNC, mci (Ross-Wang)											
Delay	Class1	0.299	0.299	1	1	0.299	0.299						
Delay	Class2	0.2495	0.2495	1.25	1.25	0.1996	0.1996						
lG = 0.5494		% log-normalization constant log g(C)											
AvgTableLDES =		% SolverLDES simulation											
Delay	Class1	0.29825	0.29932	0.99645	0.99645	0.29961	0.29932						
Delay	Class2	0.24917	0.249	1.2506	1.2506	0.19941	0.1992						

The SolverNC solver recognizes the single-Delay DROP region as a loss network and offers three solution methods: the default `exact` transform, the `erlangfp` Erlang fixed-point approximation, and `mci` Monte Carlo summation.

The admission rule analysed is $\mathbf{A}\mathbf{n} \leq \mathbf{C}$ on the per-class occupancy vector $\mathbf{n}$ of the region, and the rows of $\mathbf{A}$ are assembled from every constraint the region declares: the global job cap from `setGlobalMaxJobs`, the memory budget from `setGlobalMaxMemory` weighted by the per-class `setClassSize` footprints, the per-class caps from `setClassMaxJobs`, and any explicit linear constraint supplied through `setConstraint`. No row can distinguish stations inside the region, since the finite capacity region API expresses constraints on the per-class region occupancy alone; consequently a job moving between stations of the region leaves every left-hand side unchanged. As a cross-check, the same model is solved with SolverLDES, whose discrete-event simulator applies the DROP rule on the sample path; the simulated queue lengths and throughputs match the analytical results to within simulation error.

2.16 Tutorial 16: A queueing Petri net

A *queueing Petri net* (QPN) augments an ordinary stochastic Petri net with *queueing places* [2]: instead of a token becoming available to the output transitions immediately upon deposit, it enters a scheduling station embedded inside the place, is served under that place scheduling strategy, and only then moves to a *depository* from which the output transitions consume it. Queueing places let a single formalism capture both the contention for physical resources (through the embedded queues) and the synchronization and control flow of a Petri net (through the transitions), which is convenient when modelling, for example, software systems where requests both queue for a CPU and synchronize on shared resources.

In LINE a `Place` becomes a queueing place as soon as a service process is assigned to a token color with `set_service`; the scheduling strategy of the embedded queue is the optional third constructor argument (see the Petri-net section of the LINE user manual). This tutorial models a finite population of $N = 4$ jobs that alternate between a CPU and a think stage, each rendered as a queueing place. The CPU is a single-server FCFS queueing place with service rate 1.5; the think stage is an infinite-server queueing place with rate 0.5 (mean think time 2). Two immediate transitions move one token per firing between the two places, so the token flow reproduces a machine-repairman (finite-population) model whose exact solution is available from SolverMVA on the equivalent Delay+Queue network:

```
N = 4
model = Network('QueueingPetriNet')
# Block 1: two queueing places (set_service installs the embedded queue)
cpu = Place(model, 'CPU', SchedStrategy.FCFS) # single-server queue
think = Place(model, 'Think', SchedStrategy.INF) # infinite-server queue
jobs = ClosedClass(model, 'Jobs', N, think, 0)
cpu.set_service(jobs, Exp(1.5)) # embedded FCFS service, rate 1.5
think.set_service(jobs, Exp(0.5)) # embedded think time, mean 2
# Block 2: two immediate transitions cycling one token per firing
toCPU = Transition(model, 'toCPU')
m1 = toCPU.addMode('m1')
toCPU.setTimingStrategy(m1, TimingStrategy.IMMEDIATE)
toCPU.setEnablingConditions(m1, jobs, think, 1)
```

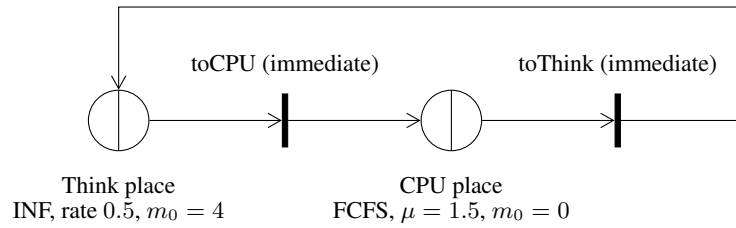


Figure 2.18: The queueing Petri net of Tutorial 16: two queueing places, drawn in the standard queueing-Petri-net shorthand as a place circle bisected by a vertical bar, exchange a single token per firing of the immediate transitions. The bar stands for the scheduling station embedded in the place, whose completions reach the depository from which the output transition consumes them; an ordinary place would be an undivided circle.

```

toCPU.setFiringOutcome(m1, jobs, cpu, 1)
toThink = Transition(model, 'toThink')
m2 = toThink.addMode('m2')
toThink.setTimingStrategy(m2, TimingStrategy.IMMEDIATE)
toThink.setEnablingConditions(m2, jobs, cpu, 1)
toThink.setFiringOutcome(m2, jobs, think, 1)
# Block 3: topology and initial marking
P = model.init_routing_matrix()
P.set(jobs, jobs, think, toCPU, 1.0)
P.set(jobs, jobs, toCPU, cpu, 1.0)
P.set(jobs, jobs, cpu, toThink, 1.0)
P.set(jobs, jobs, toThink, think, 1.0)
model.link(P)
think.set_state(N) # all tokens start in the think place
cpu.set_state(0)
# Block 4: solution. Queueing places are simulated by LDES
print(SolverLDES(model, seed=23000, samples=200000).avg_table())

```

The token counts reported for the two queueing places match the exact finite-population result to within simulation error:

AvgTableLDES =		% SolverLDES simulation of the QPN		
Station	JobClass	QLen	RespT	Tput
CPU	Jobs	1.6189	1.3626	1.1881
Think	Jobs	2.3811	2.0040	1.1881
AvgTableMVA =		% exact SolverMVA on Delay + M/M/1		
CPU	Jobs	1.6183	1.3590	1.1908
Think	Jobs	2.3817	2.0000	1.1908

The metric reported for a queueing place is its *token count*, that is the total marking summed over the tokens waiting, in service and in the depository; for the CPU place this equals the mean number in an M/M/1 queue and for the think place the mean number in service at the infinite-server stage. Queueing places are simulated by the LDES solver, which supports the FCFS, LCFS, SIRO and INF embedded scheduling strategies (single- or multi-server) with exponential, deterministic, immediate and renewal phase-type service. Because a queueing place embeds a service station inside a place, it cannot be represented as a single node in JMT; a queueing-place model is rejected by SolverJMT and must be expanded into a place plus an adjacent queueing station to be simulated there.

2.17 Tutorial 17: A colored generalized stochastic Petri net

A *generalized stochastic Petri net* (GSPN) admits two kinds of transition: *timed* ones, which fire after a random delay, and *immediate* ones, which fire as soon as they are enabled and consume no time at all [1]. It becomes *colored* when the tokens carry a type. In LINE a token color is simply a job class, so a place holds one marking per color and a transition declares one *mode* per color it serves, with its own

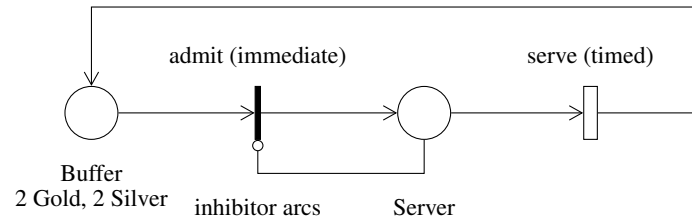


Figure 2.19: The colored GSPN of Tutorial 17. The solid thin bar is the immediate transition, the hollow bar the timed one, and the arc ending in a small circle is the inhibitor pair that keeps the server to one token: each of `admit`'s two modes is inhibited by a token of *either* color in `Server`.

firing law, enabling arcs and inhibitor arcs.

This tutorial builds the smallest net that uses all of it. Two colors, Gold and Silver, two tokens of each, circulate between a `Buffer` place and a `Server` place. The immediate transition `admit` has one mode per color and moves a waiting token into the server; both of its modes carry inhibitor arcs on *both* colors, so a token is admitted only into an empty server and the server becomes a mutual-exclusion resource. When both colors are waiting, which here is always, the conflict is resolved by the *firing weights*: Gold carries weight 2 against Silver's 1. The timed transition `serve` also has one mode per color and returns the token to the buffer at a color-dependent rate, $\mu_{\text{Gold}} = 3$ against $\mu_{\text{Silver}} = 1.5$:

```
model = Network('ColoredGSPN')
# Block 1: declare every node before parameterizing any of it
buf = Place(model, 'Buffer')
srv = Place(model, 'Server')
admit = Transition(model, 'admit')
serve = Transition(model, 'serve')
# Block 2: two token colors, both starting in the buffer
gold = ClosedClass(model, 'Gold', 2, buf, 0)
silver = ClosedClass(model, 'Silver', 2, buf, 0)
# Block 3: the immediate transition, one mode per color
mg = admit.add_mode('gold')
admit.set_timing_strategy(mg, TimingStrategy.IMMEDIATE)
admit.set_enabling_conditions(mg, gold, buf, 1)
admit.set_inhibiting_conditions(mg, gold, srv, 1)
admit.set_inhibiting_conditions(mg, silver, srv, 1)
admit.set_firing_outcome(mg, gold, srv, 1)
admit.set_firing_weights(mg, 2.0)
ms = admit.add_mode('silver')
admit.set_timing_strategy(ms, TimingStrategy.IMMEDIATE)
admit.set_enabling_conditions(ms, silver, buf, 1)
admit.set_inhibiting_conditions(ms, gold, srv, 1)
admit.set_inhibiting_conditions(ms, silver, srv, 1)
admit.set_firing_outcome(ms, silver, srv, 1)
admit.set_firing_weights(ms, 1.0)
# Block 4: the timed transition, one mode per color
sg = serve.add_mode('gold')
serve.set_distribution(sg, Exp(3.0))
serve.set_enabling_conditions(sg, gold, srv, 1)
serve.set_firing_outcome(sg, gold, buf, 1)
ss = serve.add_mode('silver')
serve.set_distribution(ss, Exp(1.5))
serve.set_enabling_conditions(ss, silver, srv, 1)
serve.set_firing_outcome(ss, silver, buf, 1)
# Block 5: topology and initial marking
P = model.init_routing_matrix()
for c in (gold, silver):
    P.set(c, c, buf, admit, 1.0)
    P.set(c, c, admit, srv, 1.0)
    P.set(c, c, srv, serve, 1.0)
    P.set(c, c, serve, buf, 1.0)
model.link(P)
buf.set_state([2, 2])
```

```

srv.set_state([0, 0])
# Block 6: exact solution, and a simulation cross-check
print(SolverCTMC(model, cutoff=4).avg_table())
print(SolverSSA(model, seed=23000, samples=200000).avg_table())

```

The immediate transition is removed before the chain is solved: stochastic complementation eliminates the vanishing markings, here 136 of them, and leaves 8 tangible states that `SolverCTMC` solves exactly. The simulation reproduces them:

AvgTableCTMC =			% exact, by stochastic complementation	
Station	JobClass	QLen	RespT	Tput
Buffer	Gold	1.5	1	1.5
Buffer	Silver	1.5	2	0.75
Server	Gold	0.5	0.33333	1.5
Server	Silver	0.5	0.66667	0.75
AvgTableSSA =			% 2e5 samples, seed 23000	
Buffer	Gold	1.5003	1.0007	1.4992
Buffer	Silver	1.4997	1.9987	0.75038
Server	Gold	0.49975	0.33333	1.4992
Server	Silver	0.50025	0.66667	0.75038

Every number here can be checked by hand, which is the point of building the net this way. Admission costs no time, so the server is never idle and $U_{\text{Gold}} + U_{\text{Silver}} = 1$. Both colors always have a token waiting, so every admission is a weighted draw and the completions split in the ratio of the firing weights, $X_{\text{Gold}} = 2X_{\text{Silver}}$. The mean service time is therefore $\frac{2}{3} \cdot \frac{1}{3} + \frac{1}{3} \cdot \frac{2}{3} = \frac{4}{9}$, so the two colors complete at $\frac{9}{4} = 2.25$ jobs per unit time in total, 1.5 Gold and 0.75 Silver, and $U_c = X_c/\mu_c = 0.5$ for each. What remains, 1.5 tokens of each color, is waiting in the buffer. Change one firing weight and the whole table moves with it: at equal weights the two colors complete at 1 each instead, and the buffer holds 1.667 Gold against 1.333 Silver.

Bibliography

- [1] M. Ajmone Marsan, G. Conte, and G. Balbo. A class of generalized stochastic petri nets for the performance evaluation of multiprocessor systems. *ACM Trans. Comput. Syst.*, 2(2):93–122, May 1984.
- [2] F. Bause. Queueing Petri nets - a formalism for the combined qualitative and quantitative analysis of systems. In *Proc. 5th Int. Workshop on Petri Nets and Performance Models (PNPM)*, pages 14–23, 1993.
- [3] D. Bini, B. Meini, S. Steffé, J. F. Pérez, and B. Van Houdt. Smcsolver and q-mam: tools for matrix-analytic methods. *SIGMETRICS Performance Evaluation Review*, 39(4):46, 2012.
- [4] A. Bobbio, A. Horváth, M. Scarpa, and M Telek. Acyclic discrete phase type distributions: properties and a parameter estimation algorithm. *Perform. Eval.*, 54(1):1–32, 2003.
- [5] G. Horváth and M. Telek. Butools 2: A rich toolbox for markovian performance evaluation. In *Proc. of VALUETOOLS*, pages 137–142, ICST, Brussels, Belgium, Belgium, 2017. ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering).
- [6] F. P. Kelly. Loss networks. *Annals of Applied Probability*, 1(3):319–378, 1991.
- [7] KT Marshall. Some relationships between the distributions of waiting time, idle time and interoutput time in the gi/g/1 queue. *SIAM Journal on Applied Mathematics*, 16(2):324–327, 1968.

Index

- activity, [27](#)
- APH distribution, [13](#)
- blocking probability, [32](#)
- breakdowns, [29](#)
- Cache node, [19](#)
- caching, [19](#)
- chain, [18](#)
- class switching, [18](#)
- ClassSwitch node, [18](#)
- closed network, [14](#)
- ClosedClass class, [14](#)
- cluster, [30](#)
- Cluster class, [31](#)
- colored Petri net, [35](#)
- CTMC solver, [13](#)
- cumulative distribution function, [23](#)
- Delay node, [14](#)
- delayed hits, [21](#)
- departure process, [25](#)
- distributions
 - APH, [13](#)
 - Erlang, [12](#)
 - Exp, [11](#)
 - Immediate, [19](#)
 - Prior, [29](#)
 - Replayer, [12](#)
 - Zipf, [19](#)
- DPS scheduling, [18](#)
- DROP, [32](#)
- entry, [27](#)
- ENV solver, [28](#)
- Environment class, [29](#)
- Erlang distribution, [12](#)
- Exp distribution, [11](#)
- FCFS scheduling, [10](#)
- finite capacity region, [32](#)
- firing mode, [35](#)
- firing weight, [36](#)
- FLD solver, [29](#)
- fluid approximation, [23](#)
- generalized stochastic Petri net, [35](#)
- GPS scheduling, [18](#)
- HOL scheduling, [18](#)
- Immediate distribution, [19](#)
- immediate feedback, [18](#)
- INF scheduling, [18](#)
- infinitesimal generator, [15](#)
- inhibitor arc, [36](#)
- insensitivity, [22](#)
- layered queueing network, [27](#)
- LayeredNetwork class, [27](#)
- LCFS scheduling, [18](#)
- LCFS-PR scheduling, [22](#)
- LDDES solver, [1](#), [10](#), [12](#), [32](#)
- LN solver, [28](#)
- load balancing, [16](#)
- loss network, [32](#)
- M/M/1 queue, [10](#)
- machine interference problem, [14](#)
- MAM solver, [13](#)
- Markov chain
 - continuous-time, [14](#)
- Marshall formula, [27](#)
- model classes
 - ClosedClass, [14](#)
 - Cluster, [31](#)
 - Environment, [29](#)
 - LayeredNetwork, [27](#)
 - OpenClass, [11](#)
- model features
 - breakdowns, [29](#)
 - caching, [19](#)
 - class switching, [18](#)
 - delayed hits, [21](#)
 - finite capacity region, [32](#)
 - immediate feedback, [18](#)

- load balancing, [16](#)
- multiclass models, [12](#)
- multiserver stations, [31](#)
- parameter uncertainty, [29](#)
- random environment, [28](#)
- synchronous calls, [27](#)
- trace-driven service, [12](#)
- warm start, [1](#), [7](#)
- multiclass models, [12](#)
- multiserver stations, [31](#)
- NC solver, [18](#)
- next reaction method, [32](#)
- node types
 - Cache, [19](#)
 - ClassSwitch, [18](#)
 - Delay, [14](#)
 - Place, [34](#)
 - Queue, [11](#)
 - Router, [16](#)
 - Sink, [11](#)
 - Source, [11](#)
 - Transition, [34](#), [35](#)
- normalizing constant, [33](#)
- OpenClass class, [11](#)
- optimization, [24](#)
- options
 - cutoff, [13](#)
 - seed, [11](#)
 - verbose, [13](#)
- parameter uncertainty, [29](#)
- percentiles, [23](#)
- performance metrics
 - Queue length, [11](#)
 - Residence time, [11](#)
 - Response time, [11](#)
 - Throughput, [11](#)
 - Utilization, [11](#)
- Place node, [34](#)
- Poisson process, [25](#)
- posterior analysis, [29](#)
- Prior distribution, [29](#)
- PROB routing, [25](#)
- product form, [32](#)
- PS scheduling, [16](#)
- Queue length (QLen), [11](#)
- Queue node, [11](#)
- queueing Petri net, [34](#)
- queueing place, [34](#)
- RAND routing, [16](#)
- random environment, [28](#)
- re-entrant line, [17](#)
- reference station, [14](#)
- reference task, [27](#)
- replacement policy
 - LRU, [19](#)
- Replayer distribution, [12](#)
- Residence time (ResidT), [11](#)
- Response time (RespT), [11](#)
- response time distribution, [23](#)
- retrieval latency, [22](#)
- retrieval system, [21](#)
- Router node, [16](#)
- routing strategies
 - PROB, [25](#)
 - RAND, [16](#)
 - RROBIN, [16](#)
- RROBIN routing, [16](#)
- sample path, [26](#)
- scheduling strategies
 - DPS, [18](#)
 - FCFS, [10](#)
 - GPS, [18](#)
 - HOL, [18](#)
 - INF, [18](#)
 - LCFS, [18](#)
 - LCFSPR, [22](#)
 - PS, [16](#)
 - SIRO, [18](#)
- service time, [10](#)
- Sink node, [11](#)
- SIRO scheduling, [18](#)
- solution methods
 - fluid approximation, [23](#)
 - Marshall formula, [27](#)
 - next reaction method, [32](#)
 - TTL algorithm, [21](#)
- solvers
 - CTMC, [13](#)
 - ENV, [28](#)
 - FLD, [29](#)
 - LDES, [1](#), [10](#), [12](#), [32](#)
 - LN, [28](#)
 - MAM, [13](#)
 - NC, [18](#)
 - SSA, [20](#)

- UQ, [29](#)
- Source node, [11](#)
- squared coefficient of variation, [12](#)
- SSA solver, [20](#)
- state space
 - generation, [15](#)
 - truncation, [13](#)
- stateful node, [26](#)
- stochastic Petri net, [34](#)
- synchronous calls, [27](#)

- Throughput (T_{put}), [11](#)
- trace-driven service, [12](#)
- Transition node, [34](#), [35](#)
- TTL algorithm, [21](#)

- UQ solver, [29](#)
- Utilization (U_{util}), [11](#)

- warm start, [1](#), [7](#)

- Zipf distribution, [19](#)